ALPE FORMACIÓN · SANTANDER

Edición 2026 / PHP 8.4 Ready

MANUAL PRÁCTICO PARA ALUMNOS

PROGRAMACIÓN EN PHP 8.x

De novato a desarrollador profesional: arquitectura web, código limpio, seguridad contra hackers, bases de datos PDO y arquitectura MVC explicados sin rodeos.

HTTP & Servidores | PHP 8.2 / 8.3 / 8.4 | MariaDB & PDO | Seguridad & CSRF | POO & Constructor Promotion | Composer & PSR-4 | MVC desde Cero | APIs REST & JSON

Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE
FORMACIÓN PROFESIONAL

A mis alumnos: Bienvenidos al oficio de programar

Carta abierta de Ramón Abramo antes de encender el editor

Queridos alumnos: si tenéis este manual en vuestras manos o en vuestras pantallas, es porque habéis tomado una de las decisiones más apasionantes de vuestra vida formativa: aprender a construir la tecnología que mueve el mundo en lugar de conformaros con ser meros espectadores.

Llevo más de 25 años entrando cada mañana en un aula de formación. He visto pasar delante de mí a miles de alumnos: algunos venían de carreras técnicas, otros de las humanidades, y muchísimos no habían escrito una sola línea de código en su vida. Muchos de ellos empezaban con el mismo miedo que quizá sentís vosotros hoy: *«Ramón, ¿seré capaz de entender esto? ¿No será la programación para matemáticos o genios de película?»*.

> *«La programación no es cuestión de magia ni de genialidad; es un oficio de constancia, lógica cotidiana y actitud. Si sabéis pedir un café con leche en un bar o seguir una receta de cocina, tenéis toda la capacidad mental necesaria para ser brillantes desarrolladores de software.»*

Este manual ha sido escrito exactamente como os hablo en clase: sin pedanterías académicas, con analogías reales (como el restaurante donde el navegador es el cliente, PHP el cocinero y la base de datos la despensa), con ejemplos prácticos línea a línea y mostrándoos sin complejos los errores que todos hemos cometido alguna vez.

PHP impulsa hoy en día más del 75% de las webs de todo el planeta. Ha madurado hasta convertirse en un lenguaje rápido, seguro, fuertemente tipado y elegante. Explotad este manual, haced que vuestro editor de código eche humo y, sobre todo, no tengáis miedo a equivocaros: cada error es simplemente un maestro más en vuestro camino.

Ramón Abramo
Docente en Alpe Formación · Santander

Créditos y Edición del Manual:
Título: Manual Práctico de Programación en PHP 8.x para Alumnos.
Autoría y Redacción Pedagógica: Ramón Abramo.
Diseño Gráfico y Maquetación: Alpe Formación · Basado en la identidad corporativa de aula y capturas de Visual Studio Code.
Versión de PHP cubierta: PHP 8.0 hasta PHP 8.4 (con soporte para MariaDB 10.x / MySQL 8.x).
Santander, Cantabria — Todos los derechos educativos reservados para los alumnos del centro.

CAPÍTULO 01 · FUNDAMENTOS Y ARQUITECTURA

¿Cómo funciona Internet y qué demonios pinta PHP?

La arquitectura cliente-servidor, el protocolo HTTP y el papel real de PHP en el desarrollo web moderno

> Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Hoy os voy a explicar este tema paso a paso. Si al principio os suena a chino, tranquilos: voy a empezar por lo que necesitáis entender y después os propongo un ejemplo para ponerlo a prueba.

Siempre que arranco un nuevo curso de desarrollo web con un grupo de alumnos, me encuentro con la misma escena en el aula: caras de expectación, personas que vienen de maquetar con HTML y CSS, y otras tantas que jamás han escrito una sola línea de código en su vida. Muchos venís con la idea preconcebida de que programar en PHP consiste simplemente en abrir un archivo con el bloc de notas, escribir cuatro cosas raras y hacer doble clic para que se abra mágicamente en Google Chrome.

¡Ni se os ocurra! Ese es el primer gran muro con el que tropieza todo principiante. Un archivo con extensión `.html` lo lee y lo dibuja vuestro propio navegador directamente desde vuestro disco duro. Pero un archivo `.php` no puede ser interpretado por vuestro navegador bajo ningún concepto. Si intentáis abrirlo con doble clic, lo único que veréis será el código fuente en bruto en la pantalla o una ventana pidiéndoos que descarguéis el archivo.

Derribando el primer bulo: «PHP está muerto»

Seguro que habéis leído en foros o redes sociales a iluminados diciendo que PHP es un lenguaje del pasado. Pues fijaos en los datos reales que monitorizan la tecnología de los servidores en todo el planeta [1]: más del **75% de todas las páginas web del mundo** con backend conocido corren sobre PHP. Plataformas gigantescas como WordPress (que mueve casi la mitad de Internet), Wikipedia o sistemas de comercio electrónico mueven miles de millones de euros diarios con PHP.

El PHP que vamos a aprender en este manual no es el lenguaje desordenado de hace quince años. Vamos a trabajar con **PHP 8.x**: un motor ultrarrápido, con tipado estricto, orientado a objetos profesional y con las mismas herramientas de ingeniería del software que cualquier otro estándar de primera línea en la industria.

> **Regla de oro del aula con Ramón**
> En mis clases no memorizamos nada sin entender el porqué. Antes de escribir código complejo, tenemos que entender con total claridad **dónde se ejecuta cada instrucción** y qué máquina es la responsable de responder. Vamos a verlo con la analogía que nunca falla.

MODELO CONCEPTUAL 01 · ENTENDER LA WEB

La gran analogía del restaurante: La web sin misterios

El comensal en la mesa, el camarero con la comanda, la cocina cerrada y la despensa bajo llave

Para entender cómo funciona cualquier página web dinámica sin caer en tecnicismos farragosos, imaginad que entráis a cenar a un restaurante bien organizado. En este restaurante intervienen cuatro actores que se coordinan a la perfección:

Figura 1.1 · Arquitectura cliente-servidor reflejada en los roles de un restaurante.

01 **El Comensal en la Mesa (Cliente / Navegador)**

Es vuestro navegador web (Chrome, Edge, Safari). Llega al restaurante, se sienta, abre la carta (la dirección URL) y pide lo que quiere. El cliente jamás entra a la cocina ni toca las sartenes; solo sabe comerse el plato cuando se lo sirven terminado.

02 **El Camarero (Protocolo HTTP)**

Es el mensajero que conecta la sala con la cocina. Recoge la comanda de vuestra mesa (`HTTP Request`), la lleva a la cocina y, cuando el plato está cocinado, os lo trae de vuelta a la mesa (`HTTP Response`).

03 **El Chef en la Cocina (Servidor Web + PHP)**

Aquí está el meollo. El servidor web (Apache o Nginx) recibe la comanda y se la pasa al chef (el intérprete de PHP). El chef lee la receta (vuestro código PHP), procesa los ingredientes, mezcla datos y prepara el plato final en formato HTML estándar.

04 **La Despensa con Candado (MySQL / MariaDB)**

Es el almacén de datos. Solo el chef tiene la llave para entrar a por ingredientes frescos (usuarios, contraseñas, productos). El comensal tiene terminantemente prohibido el paso a la despensa por motivos evidentes de seguridad.

FUNDAMENTO TÉCNICO 02 · DOS MUNDOS DISTINTOS

Cliente vs. Servidor: La frontera infranqueable

Por qué el navegador jamás ve código PHP y qué ocurre realmente en cada máquina

Vamos a separar de una vez por todas lo que pasa en el ordenador del usuario de lo que pasa en la máquina remota donde reside la web. Si tenéis clara esta distinción, os ahorraréis el 90% de los errores de novato en vuestro código.

El lado del cliente (Frontend)

El cliente es el dispositivo del usuario: su portátil, su móvil o su tableta. En él se ejecuta el navegador web. Los navegadores solo entienden tres lenguajes nativos: **HTML** (que define la estructura del documento), **CSS** (que define los colores, fuentes y maquetación visual) y **JavaScript** (que añade dinamismo e interactividad visual en la propia ventana).

El lado del servidor (Backend)

El servidor es un ordenador (habitualmente en un centro de datos conectado permanentemente a Internet) con un software especializado escuchando peticiones. En este servidor es donde vive nuestro código PHP. Cuando el camarero le entrega la petición, el intérprete de PHP lee el archivo de arriba abajo, ejecuta bucles, consulta bases de datos, comprueba contraseñas y **genera un resultado**.

> **Experimento de aula: Inspeccionando el código fuente**
> Abrid cualquier página web en vuestro navegador habitual (por ejemplo, la portada de Wikipedia o un periódico digital) y pulsad la combinación de teclas `Ctrl + U` (o clic derecho > *Ver código fuente de la página*). Fijaos en todo el texto que aparece: veréis miles de líneas de HTML, estilos CSS y scripts de JavaScript. Pero **no encontraréis ni una sola etiqueta de PHP** en todo el documento. ¿Por qué? Porque PHP ya ha hecho su trabajo en el servidor y se ha evaporado. Al cliente solo llega el plato cocinado.

Resumen de diferencias entre entornos

Característica	Lado del Cliente (Frontend)	Lado del Servidor (Backend con PHP)
¿Dónde se ejecuta?	En el navegador del usuario (Chrome, Edge, Firefox).	En el servidor web remoto (o en Laragon en tu máquina local).
Lenguajes que procesa	HTML5, CSS3, JavaScript nativo.	PHP 8.x, SQL (MySQL/MariaDB), Python, etc.
Visibilidad del código	100% público. Cualquier usuario puede ver tu HTML y tu JS.	100% privado. El usuario jamás tiene acceso a tus archivos PHP.
Acceso a base de datos	Ninguno de forma directa (sería un suicidio de seguridad).	Acceso total y controlado mediante consultas preparadas con PDO.

PROTOCOLO OFICIAL · LA VÍA DE COMUNICACIÓN

El protocolo HTTP al desnudo: Peticiones y Respuestas

Cómo viaja la información por la red: cabeceras, cuerpo de mensaje y la gran batalla entre GET y POST

El camarero de nuestro restaurante tiene un nombre oficial: **HTTP** (*HyperText Transfer Protocol*, Protocolo de Transferencia de Hipertexto) [2]. Es el conjunto de reglas que permite que un cliente y un servidor se entiendan sin equivocarse.

1. Anatomía de una Petición (`HTTP Request`)

Cuando escribís una dirección web o pulsáis un enlace, vuestro navegador construye un paquete de texto plano estructurado en tres partes:

Línea de Petición
Indica el método (ej. `GET`), la ruta solicitada (`/catalogo.php`) y la versión (`HTTP/1.1`).

Cabeceras (Headers)
Metadatos del cliente: qué navegador usa (`User-Agent`), qué idioma prefiere (`Accept-Language`) o cookies activas.

Cuerpo (Body)
Los datos que enviáis. En una petición `GET` suele ir vacío; en un `POST` contiene el formulario con los datos.

2. La gran batalla de métodos: GET vs. POST

En PHP nos vamos a hartar de trabajar con las superglobales `$_GET` y `$_POST`. Fijaos bien en sus diferencias para no meter la pata:

Criterio	Método GET	Método POST
¿Dónde viajan los datos?	Visibles en la propia barra de direcciones URL tras el signo `?` (`buscar.php?q=zapatos&talla=42`).	Ocultos dentro del cuerpo (`Body`) de la petición HTTP. No se ven en la barra de direcciones.
Límite de tamaño	Muy limitado (alrededor de 2.048 caracteres según el navegador).	Prácticamente ilimitado (configurable en `php.ini` con `post_max_size`).
Historial y marcadores	Se guarda en el historial de navegación y se puede añadir a favoritos o compartir el enlace.	No se guarda en marcadores. Si recargáis la página, el navegador os avisará: *«¿Desea reenviar el formulario?»*.
Uso recomendado	**Consultas y búsquedas** donde no se altere el estado del servidor.	**Altas, modificaciones, login** y subida de archivos confidenciales.

⚠️ **Error garrafal: Enviar contraseñas por GET**

En mis más de 25 años dando clase he visto a alumnos mandar el formulario de registro o de inicio de sesión con el método `GET` . Eso significa que la contraseña del usuario viaja visible en la barra de direcciones del navegador, se queda guardada en el historial de navegación, en los registros del proxy y a la vista de cualquiera que mire la pantalla por encima del hombro. **Los datos sensibles siempre van por POST**.

DIAGNÓSTICO TÉCNICO · HABLANDO CON EL SERVIDOR

Códigos de estado HTTP: El semáforo de Internet

Qué nos dice el servidor en cada respuesta: del éxito rotundo al desastre en la cocina

Cuando el chef termina de procesar la comanda, el camarero vuelve trayendo una respuesta (`HTTP Response`). Cada respuesta incluye obligatoriamente un **código de estado numérico de tres dígitos** [2]. Estos códigos están clasificados por familias según su primer dígito:

Familia	Significado en el aula	Códigos imprescindibles	Qué ha ocurrido realmente
1xx	Informativos	`100 Continue`	Petición recibida, continúa con el proceso. Muy raro que lo manejéis en el día a día.
2xx	¡Éxito rotundo!	`200 OK` `201 Created`	El servidor ha procesado la petición sin ningún contratiempo y os entrega los datos solicitados (el plato en vuestra mesa).
3xx	Redirecciones	`301 Moved` `302 Found`	El recurso que buscas ya no está en esta dirección; te mudo automáticamente a la nueva ubicación.
4xx	Error del cliente	`400 Bad Request` `401 Unauthorized` `403 Forbidden` `404 Not Found`	La culpa es tuya: has escrito mal la URL, has pedido una página que no existe o estás intentando entrar a un área protegida sin iniciar sesión con tu usuario.
5xx	Error del servidor	`500 Internal Error` `502 Bad Gateway` `503 Unavailable`	La cocina está ardiendo: tu script PHP tiene un error de sintaxis fatal que no has capturado, la base de datos se ha caído o el servidor se ha quedado sin memoria.

Cómo ver las cabeceras reales con DevTools

No os creáis nada de lo que os digo sin comprobarlo con vuestros propios ojos. Cualquier navegador moderno tiene un inspector de red profesional integrado.

1. **Abrid las Herramientas de Desarrollador**
   En vuestro navegador, pulsad la tecla `F12` o `Ctrl + Shift + I` .
2. **Id a la pestaña Red (Network)**
   Haced clic en la pestaña **Red (Network)** y aseguraos de que el filtro esté en *«Todo»* o *«Doc»*.
3. **Recargad la página y pulsad en la primera petición**
   Pulsad `F5` . Veréis aparecer la lista de archivos que viajan por el cable. Haced clic sobre el primero: veréis las cabeceras de petición (`Request Headers`) y las cabeceras de respuesta (`Response Headers`) con su código 200 en verde.

TALLER PRÁCTICO 01 · MANOS A LA OBRA

Práctica guiada: Manipulando cabeceras con PHP

Cómo forzar una respuesta en JSON, cambiar códigos de estado y evitar el bloqueo por envío previo

Para consolidar lo aprendido, vamos a escribir un script PHP en Visual Studio Code que altere deliberadamente las cabeceras que el servidor envía de vuelta al cliente mediante la función nativa `header()` [3].

cabeceras_demo.php — curso-php — Visual Studio Code

cabeceras_demo.php

```php
<?php
declare(strict_types=1);

// 1. Le decimos al navegador que la respuesta no es HTML, sino JSON
header('Content-Type: application/json; charset=utf-8');

// 2. Modificamos el código de estado a 201 Created
http_response_code(201);

// 3. Preparamos los datos en un array asociativo
$respuesta = [
    'aula' => 'Alpe Formación',
    'profesor' => 'Ramón Abramo',
    'estado' => 'Petición procesada con éxito',
    'fecha' => date('Y-m-d H:i:s')
];

// 4. Transformamos el array a formato JSON y lo enviamos
echo json_encode($respuesta, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);
```

http://localhost/cabeceras_demo.php — 201 CREATED

```
{
    "aula": "Alpe Formación",
    "profesor": "Ramón Abramo",
    "estado": "Petición procesada con éxito",
    "fecha": "2026-11-04 16:30:00"
}
```

⚠️ El error número 1 de PHP: «Headers already sent»

Si ponéis un `echo`, un espacio en blanco o una sola etiqueta HTML **antes** de la función `header()`, PHP os soltará el temido aviso en rojo: *«Cannot modify header information - headers already sent»*. El camarero no puede entregar la comanda cuando ya ha llevado los platos a la mesa. Las cabeceras siempre van antes que cualquier salida.

CONSOLIDACIÓN DEL APRENDIZAJE

Comprobación de conocimientos del Capítulo 1

Cinco preguntas clave de aula para verificar que los cimientos están listos para el Capítulo 2

1. ¿Por qué al hacer doble clic sobre un archivo .php en el explorador de Windows no se ejecuta el código?

> Porque el navegador carece de intérprete de PHP integrado. Los archivos PHP requieren obligatoriamente pasar por un servidor web (como Apache o Nginx) que procese el script y devuelva HTML puro al cliente.

2. En la analogía del restaurante, ¿qué componente representa la base de datos MySQL y quién puede acceder a ella?

> Representa la despensa cerrada con candado. El único con llave para entrar a por ingredientes es el chef (el servidor con PHP). El comensal (cliente) jamás puede acceder directamente a la despensa.

3. ¿Qué diferencia práctica fundamental existe entre los métodos HTTP GET y POST al enviar un formulario?

> El método GET viaja visible en la barra de direcciones de la URL y tiene límite de longitud, ideal para búsquedas. El método POST viaja oculto dentro del cuerpo de la petición sin límite estricto, imprescindible para contraseñas y registros.

4. Si tu servidor responde con un código de estado 404 y otro día con un 500, ¿dónde está el fallo en cada caso?

> El 404 es un error del cliente (el archivo o recurso solicitado no existe o la URL está mal escrita). El 500 es un error grave en la cocina del servidor (tu código PHP contiene un fallo fatal no controlado o la base de datos no responde).

5. ¿Por qué la función header() debe ejecutarse siempre antes de cualquier instrucción echo?

> Porque en el protocolo HTTP las cabeceras viajan antes que el cuerpo del mensaje. Si envías aunque sea un solo espacio de texto, el servidor ya ha emitido las cabeceras por defecto y bloqueará cualquier modificación posterior («headers already sent»).

Lo que debéis tener claro antes de abrir el Capítulo 2

Con estos conceptos afianzados en la cabeza, ya sabéis exactamente qué papel juega PHP en el ecosistema web. En el **Capítulo 2** nos meteremos de lleno en el taller práctico: instalaremos y configuraremos **Laragon** en Windows, analizaremos sus diferencias con ServBay y XAMPP, y configuraremos Visual Studio Code con las extensiones que usan los profesionales.

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *W3Techs - Usage Statistics and Market Share of Server-side Programming Languages for Websites*: https://w3techs.com/technologies/overview/programming_language
[2] *RFC 9110 - HTTP Semantics (IETF / RFC Editor)*: https://www.rfc-editor.org/rfc/rfc9110.html
[3] *PHP Manual - Función header() y gestión de cabeceras HTTP*: https://www.php.net/manual/es/function.header.php

CAPÍTULO 02 · EL TALLER DE TRABAJO

Montando el entorno de desarrollo perfecto

Instalación paso a paso de Laragon, comparativa real frente a ServBay y XAMPP, y configuración pro de VS Code

> Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Hoy vamos a mancharnos las manos montando vuestro taller de trabajo. Si queréis ser buenos desarrolladores web, vuestro ordenador tiene que estar configurado con herramientas profesionales que os ahorren tiempo, no que os den disgustos.

En el capítulo anterior dejamos claro que el navegador web del usuario no tiene la menor idea de cómo procesar un archivo con extensión `.php`. Para poder trabajar en vuestro ordenador particular sin necesidad de contratar un hosting en Internet cada vez que queráis probar una línea de código, necesitamos simular un servidor completo dentro de vuestro propio sistema operativo.

La «Trinidad» del desarrollo web backend

Para que vuestro ordenador funcione como un servidor de desarrollo completo, necesitamos exactamente tres piezas de software coordinadas entre sí:

1. Servidor Web (Apache)
Escucha las peticiones entrantes en el puerto 80 o 8080, localiza los archivos en vuestro disco duro y gestiona el tráfico HTTP de la red.

2. Intérprete (PHP 8.x)
El motor que lee el código fuente de vuestros scripts, ejecuta la lógica, procesa las variables y genera la salida en HTML o JSON.

3. Base de Datos (MySQL)
El gestor relacional (habitualmente MariaDB) donde almacenaremos usuarios, pedidos y tablas con total persistencia.

Podríais intentar descargar e instalar estas tres herramientas por separado, compilar Apache a mano, configurar las variables de entorno de Windows y editar archivos de texto kilométricos. Pero os aseguro que en mis clases ningún alumno pierde dos días peleando con la consola cuando existen suites modernas integradas que resuelven el montaje en cinco minutos.

> **Filosofía docente: El taller debe ser invisible**
> El objetivo de este curso es que aprendáis a programar y a resolver problemas con PHP, no que os convirtáis en administradores de sistemas frustrados. La mejor herramienta es la que arranca con un solo clic y no se interpone entre vuestras ideas y la pantalla.

PASO A PASO GUIADO · WINDOWS 10 / 11

Instalando Laragon: La navaja suiza moderna

Por qué es la elección número uno en nuestras aulas y cómo configurarlo limpiamente desde cero

Para los que trabajáis con Windows (que sois la inmensa mayoría en el aula), mi recomendación indiscutible es **Laragon** [1]. Es ligero, no ensucia el registro del sistema operativo, incluye terminal con Git y, lo más importante, gestiona dominios virtuales automáticos sin tocar el archivo hosts a mano.

Los cuatro pasos para una instalación perfecta

1. **Descarga el paquete «Laragon Full»**
   Entrad en la web oficial `laragon.org/download` y elegid la edición **Laragon - Full (64-bit)**. Esta versión ya viene con PHP 8.x, Apache 2.4, MariaDB 10.x, Node.js y Git integrados.
2. **Ruta raíz: C:\laragon (¡Sin espacios!)**
   Durante el instalador, elegid como ruta de destino `C:\laragon`. **Ni se os ocurra** instalarlo dentro de `C:\Archivos de Programa` ni en carpetas con nombres compuestos con tildes. Las rutas con espacios en blanco son el veneno de los comandos de terminal.
3. **Activar Virtual Hosts automáticos**
   En las opciones finales de la instalación, dejad marcada la casilla *«Auto Virtual Hosts»* y el sufijo `.test`. Esto creará automáticamente dominios locales seguros como `http://tienda.test` para cada carpeta que creéis.
4. **Arranque de servicios con «Start All»**
   Abrid la ventana de Laragon y pulsad el botón grande que dice **«Start All»**. Si Windows Defender os muestra un aviso de cortafuegos para Apache y MariaDB, marcad las casillas de redes privadas y pulsad *«Permitir acceso»*.

La anatomía de las carpetas de Laragon

Carpeta en C:\laragon	Qué contiene	Cómo interactuamos los desarrolladores con ella
`/www/`	La raíz de todos vuestros proyectos web.	**Aquí es donde trabajaréis todos los días**. Cada subcarpeta que creéis aquí será una web independiente.
`/bin/php/`	Las versiones de PHP instaladas.	Podéis descargar versiones nuevas de PHP (8.2, 8.3, 8.4), descomprimirlas aquí y cambiar de una a otra con el botón derecho.
`/bin/apache/`	El servidor web Apache y sus módulos.	Aquí residen los archivos de configuración `httpd.conf` y los hosts virtuales.
`/bin/mysql/`	El motor de base de datos MariaDB/MySQL.	Almacena los datos y el motor SQL sobre el que correrán vuestras tablas.

ANÁLISIS TÉCNICO · LA DECISIÓN CORRECTA

Laragon vs. ServBay vs. XAMPP: La comparativa

Diferencias clave, ventajas, desventajas y qué herramienta elegir según vuestro sistema operativo

En mis clases siempre hay alumnos con ordenadores Mac o personas que aprendieron hace años con XAMPP y se preguntan por qué insisto en renovar las herramientas. Vamos a analizar cada alternativa con total honestidad profesional:

Entorno	Plataforma ideal	Puntos fuertes (Ventajas)	Puntos débiles (Desventajas)	Veredicto de Ramón
Laragon	Windows 10 / 11	VirtualHosts automáticos con `.test`, portable, cambio de versión de PHP en 2 segundos, terminal Zsh integrado, extremadamente rápido.	No tiene versión nativa para macOS ni Linux. Solo existe para Windows.	**La opción Nº1 indiscutible** para cualquier alumno con PC Windows.
ServBay	macOS (Apple Silicon M1/M2/M3 e Intel)	Interfaz moderna para Mac, gestión de múltiples versiones de PHP simultáneas sin reiniciar, MariaDB, PostgreSQL y Redis nativos en un clic.	Relativamente reciente en el mercado; algunas funciones empresariales avanzadas requieren plan de pago.	**La mejor opción para Mac**. Olvidaos de pelear con Homebrew si estáis empezando.
XAMPP	Multiplataforma (Win, Mac, Linux)	El más veterano y conocido. Hay millones de tutoriales antiguos en Internet resolviendo dudas clásicas.	Cambiar de versión de PHP es una pesadilla; no tiene VirtualHosts automáticos; su panel de control es arcaico y propenso a bloqueos.	**Superado**. Válido si estás obligado por un centro antiguo, pero prescindible hoy.
PHP Built-in Server	Cualquier consola de comandos	No requiere instalar ninguna suite pesada. Solo abres el terminal y escribes `php -S localhost:8000`.	Monohilo (atiende una sola petición a la vez); no incluye MySQL ni soporte para archivos `.htaccess`.	**Excelente para pruebas rápidas** de algoritmos y scripts de una sola hoja.

¿Y qué pasa con Docker?

Muchos desarrolladores veteranos os dirán: *«¡Usa contenedores Docker con DDEV o Laravel Sail!»*. Y tienen razón... **cuando ya sabéis programar**. Pero meter a un alumno que aún no sabe qué es una variable a pelearse con imágenes de Linux, puertos mapeados en Docker Desktop y archivos YAML de composición es la receta perfecta para que abandone el curso en la primera semana. Primero dominamos el lenguaje; los contenedores vendrán más adelante.

> **Solución rápida: El conflicto del puerto 80 ocupado**
> Si al pulsar *«Start All»* en Laragon o XAMPP el botón de Apache se apaga o muestra letras rojas, el 99% de las veces es porque el puerto 80 está secuestrado por Skype, IIS o los servicios de SQL Server. La solución más limpia: en Laragon haced clic derecho > *Preferencias > Servicios y Puertos* y cambiad el puerto de Apache de 80 a **8080**. Vuestras webs funcionarán entrando por `http://localhost:8080` sin que nada interfiera.

TU EDITOR PROFESIONAL · PRODUCTIVIDAD

Configuración de Visual Studio Code para PHP

Extensiones imprescindibles, autocompletado inteligente y vinculación con el ejecutable de PHP

Programar en el Bloc de notas o en editores sin soporte para PHP es como intentar construir un mueble usando una piedra como martillo. **Visual Studio Code (VS Code)** es el editor estándar del sector: es gratuito, de código abierto, ultrarrápido y tiene un ecosistema de extensiones inigualable [2].

Las 4 extensiones que tenéis que instalar sí o sí

01 PHP Intelephense (de Ben Mewburn)
La extensión reina. Añade autocompletado inteligente de funciones, detección de errores de sintaxis en tiempo real, navegación a definiciones y análisis estático de tipos sin consumir apenas recursos.

02 PHP Debug (de Xdebug)
Permite conectar VS Code con el depurador Xdebug de PHP para pausar la ejecución en un punto de interrupción (breakpoint), inspeccionar variables vivas y cazar fallos sin plagar el código de `echo`.

03 PHP CS Fixer
Formatea vuestro código automáticamente según los estándares profesionales PSR-12 cada vez que pulsáis `Ctrl + S`. Adiós a sangrías torcidas y llaves mal alineadas.

04 Material Icon Theme
Asigna a cada archivo un icono claro y visual según su extensión (el elefante azul para `.php`, el escudo naranja para `.html`, etc.). Ayuda a no perderse en proyectos grandes.

Vinculando VS Code con el PHP de Laragon

Para que VS Code pueda validar la sintaxis de vuestro código mientras escribís, necesita saber exactamente dónde está el ejecutable `php.exe` en vuestro ordenador.

settings.json — Configuración de Usuario — VS Code

settings.json

```
{
    "php.validate.executablePath": "C:\\laragon\\bin\\php\\php-8.3.4-Win32-vs16-x64\\php.exe",
    "php.suggest.basic": false,
    "editor.tabSize": 4,
    "editor.formatOnSave": true
}
```

Atajo de aula: Cómo abrir settings.json en un segundo
En VS Code pulsad `Ctrl + Shift + P` para abrir la paleta de comandos, teclead *«Open User Settings (JSON)»* y pulsad Intro. Pegad la ruta de vuestro `php.exe` tal como se muestra en la captura superior (fijaos en que las barras van duplicadas `\\`).

TALLER PRÁCTICO 02 · VERIFICACIÓN

Tu primer proyecto y la radiografía con phpinfo()

Creación de la carpeta de trabajo, comprobación de módulos y directivas clave en php.ini

Con Laragon funcionando y VS Code configurado, ha llegado el momento de crear vuestro primer proyecto real y realizar una radiografía completa del motor de PHP mediante la función oficial `phpinfo()` [3].

1. **Crear la carpeta del proyecto en www**
   Abrid el explorador de archivos e id a `C:\laragon\www`. Cread una carpeta llamada `verificacion`.
2. **Abrir la carpeta en VS Code**
   Abrid VS Code, pulsad *Archivo > Abrir carpeta* y seleccionad `C:\laragon\www\verificacion`.
3. **Crear el archivo info.php**
   Cread un archivo nuevo llamado `info.php` y escribid exactamente el siguiente código:

info.php — verificacion — Visual Studio Code

info.php

```php
<?php
// Este script genera el informe técnico completo de nuestro servidor
phpinfo();

```

Directivas críticas de php.ini que debéis vigilar

Al abrir en vuestro navegador `http://localhost/verificacion/info.php` veréis una tabla gigante morada con cientos de configuraciones. Estas son las cuatro directivas que en clase siempre revisamos en el archivo `php.ini`:

Directiva en php.ini	Valor ideal para aprender	Por qué es tan importante
`display_errors`	`On`	Si está en *Off* y cometéis un error de sintaxis, la pantalla se quedará en blanco sin deciros qué ha fallado. En desarrollo debe estar siempre en **On**.
`error_reporting`	`E_ALL`	Obliga a PHP a avisarnos de cualquier fallo, incluyendo advertencias menores y variables no inicializadas.
`memory_limit`	`256M` o `512M`	La memoria RAM máxima que puede consumir un script antes de que el servidor lo mate.
`upload_max_filesize`	`64M`	El peso máximo de archivos que podéis subir mediante formularios HTML. El valor por defecto de 2M se queda corto enseguida.

⚠️ Peligro de novato: Dejar info.php en un servidor real

El archivo `phpinfo()` es fantástico para comprobar vuestro ordenador en casa. Pero **jamás lo subáis a un servidor en producción en Internet**. Ese archivo le muestra a cualquier atacante la versión exacta de PHP, las rutas de vuestros discos duros y los módulos instalados. Borradlo siempre antes de publicar.

CONSOLIDACIÓN DEL TALLER

Comprobación de conocimientos del Capítulo 2

Cinco preguntas clave para certificar que vuestro entorno está a punto para empezar a programar

1. ¿Qué tres componentes indispensables forman la arquitectura de desarrollo backend local?

> El Servidor Web (Apache o Nginx), el Intérprete de lenguaje (motor de PHP 8.x) y el Gestor de Base de Datos relacional (MySQL o MariaDB).

2. ¿Por qué se desaconseja totalmente instalar Laragon o herramientas similares en carpetas con espacios como «Archivos de Programa»?

> Porque los comandos internos de consola, terminales Git y scripts de automatización interpretan los espacios como separadores de argumentos, provocando fallos de ejecución constantes.

3. Si al arrancar Apache en Laragon aparece un fallo por el puerto 80 ocupado, ¿cuál es la solución más rápida sin desinstalar nada?

> Cambiar en las preferencias de Laragon el puerto de Apache a 8080. A partir de ese momento se accede a las webs locales mediante la dirección http://localhost:8080.

4. ¿Qué ventaja sustancial aporta la extensión «PHP Intelephense» frente a escribir código en un editor de texto convencional?

> Ofrece autocompletado inteligente de sintaxis, análisis estático para detectar fallos antes de ejecutar y documentación emergente de parámetros para todas las funciones nativas de PHP.

5. ¿Por qué la directiva display_errors del archivo php.ini debe estar en «On» en nuestro taller de aprendizaje?

> Porque si estuviera en «Off», cualquier error sintáctico o fatal generaría una pantalla completamente en blanco («pantalla blanca de la muerte»), impidiéndonos saber qué línea ha fallado.

Lo que nos espera en el Capítulo 3

¡Enhorabuena, equipo! Con el taller montado y los motores en marcha, ya tenemos el escenario listo para lo bueno. En el **Capítulo 3** entraremos de lleno en la materia prima del programador: **Sintaxis básica, variables, tipos de datos escalares, tipado estricto y constantes mágicas**.

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *Laragon Official Documentation - Getting Started & Auto Virtual Hosts*: https://laragon.org/docs/
[2] *Visual Studio Code - PHP Programming Documentation & Intelephense*: https://code.visualstudio.com/docs/languages/php
[3] *PHP Manual - Función phpinfo() y configuración del núcleo*: https://www.php.net/manual/es/function.phpinfo.php
[4] *ServBay Documentation - Development Environment on macOS*: https://www.servbay.com/docs

CAPÍTULO 03 · FUNDAMENTOS DEL LENGUAJE

Sintaxis básica, variables y tipos de datos

Las etiquetas maestras, el símbolo del dólar, los tipos primitivos y la directiva strict_types

Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Hoy entramos en la auténtica cocina de PHP. Se acabaron los preliminares y las instalaciones: a partir de este momento vamos a aprender las palabras, los ladrillos y las reglas con las que se construye cualquier aplicación web.

Todo lenguaje de programación tiene una gramática. Si en español os dejáis los puntos o cambiáis el orden de las palabras, la frase se vuelve confusa. Pero el ordenador es un comensal infinitamente más tiquismiquis: **si olvidáis un punto y coma o abrís unas comillas que luego no cerráis, el intérprete se detendrá en seco** y os mostrará un error en rojo sin pestañear.

1. Las etiquetas de apertura y la norma de cierre

Para que el servidor web sepa qué fragmento de un archivo debe ser cocinado por el motor de PHP, utilizamos la etiqueta de apertura oficial [1]:

<?php **Etiqueta de apertura oficial**

Indica al motor: «¡Despierta, chef! A partir de aquí todo lo que viene son órdenes en PHP que debes ejecutar en memoria».

?> **Etiqueta de cierre (¡Ojo con esto!)**

En archivos que contienen únicamente código PHP, **la norma profesional prohíbe cerrar la etiqueta**. Dejad el archivo abierto.

Por qué nunca cerramos con ?> en archivos puros de PHP

Si ponéis `?>` al final de un script y se os escapa un espacio en blanco o un salto de línea accidental detrás de él, el servidor interpretará ese espacio invisible como contenido HTML y lo enviará al cliente. Si más adelante intentáis enviar una cookie o una cabecera con `header()`, el script reventará con el error *«headers already sent»*. Omitir el cierre es el estándar PSR oficial.

2. Variables y la metáfora de las cajas de zapatos

Una variable es sencillamente una **caja etiquetada en la memoria RAM** del servidor donde guardamos un dato para reutilizarlo tantas veces como queramos. En PHP, todas las variables empiezan obligatoriamente por el símbolo del dólar: `$`.

SISTEMAS DE TIPOS · RIGOR PROFESIONAL

Los tipos primitivos y declare(strict_types=1)

Enteros, decimales, textos, booleanos y cómo evitar que PHP transforme datos al azar

Históricamente, PHP era un lenguaje de tipado débil que intentaba «adivinar» qué queríais hacer. Si sumabais el texto `"5"` y el número `10`, PHP lo convertía por detrás y devolvía `15`. Esto, que a priori parecía muy cómodo para neófitos, provocaba desastres monumentales en aplicaciones financieras y de comercio electrónico.

Los cuatro tipos de datos escalares esenciales

Tipo técnico	Nombre en PHP	Ejemplo real	Para qué se utiliza en una aplicación
Entero	`int`	`$edad = 25;`	Contadores, identificadores de base de datos, cantidades de stock y años. No admiten decimales.
Coma flotante	`float`	`$precio = 19.99;`	Precios, porcentajes de IVA, distancias y cálculos científicos con decimales (separados por punto).
Cadena de texto	`string`	`$nombre = "Ramón";`	Textos, correos electrónicos, párrafos y códigos alfanuméricos entre comillas.
Booleano	`bool`	`$matriculado = true;`	Banderas lógicas que solo admiten dos valores posibles: `true` (verdadero) o `false` (falso).

El candado de seguridad: declare(strict_types=1)

En mis clases siempre os obligo a poner esta instrucción como primera línea de todos vuestros archivos PHP [2]:

tipos_estrictos.php — Visual Studio Code

tipos_estrictos.php

```php
<?php
declare(strict_types=1);

// Si strict_types está activo, pasar un string donde se espera un int genera un TypeError fatal
function calcularTotal(int $cantidad, float $precio): float {
    return $cantidad * $precio;
}
```

Por qué os exijo tipado estricto desde el primer día

Porque os educa como programadores profesionales. Si una función espera un número entero y le pasáis una palabra, queremos que el programa falle de inmediato en vuestro ordenador de prácticas, no meses después cuando esté cobrando dinero real a un cliente en producción.

MANIPULACIÓN DE TEXTO · COMILLAS

Cadenas: Comillas simples vs. dobles y Heredoc

Interpolación de variables, secuencias de escape y bloques de texto HTML multilínea

El 80% de lo que hace una aplicación web con PHP es formatear texto para el comensal. Por eso, dominar cómo se delimitan las cadenas de texto es imprescindible.

Comillas simples (Texto literal puro)

PHP no busca nada dentro. Lo que escribís es exactamente lo que sale. Si escribís `'$nombre'`, mostrará literalmente el símbolo del dólar y la palabra nombre. Son ligeramente más rápidas.

Comillas dobles (Interpolación activa)

PHP analiza el interior. Si encuentra una variable como `"Hola {$nombre}"`, sustituye automáticamente la variable por su valor real en memoria. Además interpreta secuencias como `\n`.

cadenas_demo.php — Visual Studio Code

cadenas_demo.php

```php
<?php
$alumno = "Laura";
$modulo = "PHP 8";

// 1. Comillas simples: muestra literal
echo 'Bienvenida $alumno al curso de $modulo';
// Salida: Bienvenida $alumno al curso de $modulo

// 2. Comillas dobles con llaves (la forma recomendada por Ramón):
echo "Bienvenida {$alumno} al curso de {$modulo}";
// Salida: Bienvenida Laura al curso de PHP 8

// 3. Concatenación manual con el punto (.):
echo "Bienvenida " . $alumno . " al aula.";
```

Sintaxis Heredoc: Cuando el HTML se vuelve gigante

¿Qué pasa cuando queréis emitir un bloque de diez líneas de HTML con enlaces, clases y botones? Escapar comillas con barras invertidas `"` es una pesadilla ilegible. Para eso existe la sintaxis **Heredoc** [1]:

heredoc_ejemplo.php — Visual Studio Code

heredoc_ejemplo.php

```php
$tarjetaHtml = <<<HTML
<div class="card">
    <h2>{$alumno}</h2>
    <p>Matriculada en {$modulo} con el profesor Ramón Abramo.</p>
</div>
HTML;
```

VALORES INMUTABLES · SEGURIDAD

Constantes: define(), const y constantes mágicas

Diferencias clave, mayúsculas normativas y las variables del sistema __DIR__ y __FILE__

Si una variable es una caja cuyo contenido puede cambiar a lo largo del programa, una **constante** es una caja grabada en piedra: una vez que le asignáis un valor al inicio, **nada ni nadie puede alterarlo** durante la ejecución del script.

1. Dos formas de declarar constantes: const vs. define()

Mecanismo	Sintaxis en código	Momento de evaluación	Dónde puede usarse
`const`	`const IVA = 0.21;`	En tiempo de compilación (más rápida y legible).	Ámbito global y dentro de clases orientadas a objetos.
`define()`	`define('API_KEY', 'abc123xyz');`	En tiempo de ejecución.	Permite nombres condicionales o dentro de bloques `if`.

2. Las constantes mágicas de PHP

PHP incluye un conjunto especial de constantes predefinidas que cambian de valor según el lugar exacto del código donde las invoquéis [3]. Se reconocen al instante porque empiezan y terminan con dos guiones bajos:

magicas_demo.php — Visual Studio Code

magicas_demo.php

```php
<?php
echo "Estamos ejecutando la línea: " . __LINE__ . "<br>";
echo "El archivo actual es: " . __FILE__ . "<br>";
echo "La carpeta física en disco es: " . __DIR__ . "<br>";


```

La regla de oro de las rutas con __DIR__

En mis clases jamás os permitiré incluir archivos con rutas relativas como `include '../../config.php'`. Si movéis el archivo de carpeta, el script se romperá. Acostumbraos desde hoy a construir rutas absolutas e indestructibles utilizando siempre `__DIR__ . '/config.php'`.

HERRAMIENTAS DE DEPURACIÓN · DIAGNÓSTICO

Abriendo las tripas de una variable: var_dump()

Por qué echo se queda corto y cómo inspeccionar el tipo y tamaño exacto en memoria

Muchos alumnos se frustran cuando una variable booleana `$activo = false;` no muestra nada en pantalla al hacer `echo $activo;`. Y es que `echo` solo sabe convertir cosas a texto para el usuario final. Cuando los programadores queremos saber qué hay realmente dentro de una caja en memoria, usamos **var_dump()** [4].

vardump_demo.php — Visual Studio Code

vardump_demo.php

```php
<?php
$entero = 42;
$decimal = 3.1416;
$texto = "Alpe Formación";
$booleano = false;
$nulo = null;

// var_dump muestra: tipo, tamaño en caracteres o bytes, y valor real
var_dump($entero, $decimal, $texto, $booleano, $nulo);
```

http://localhost/vardump_demo.php

```
int(42)
float(3.1416)
string(15) "Alpe Formación"
bool(false)
NULL
```

Los tres errores típicos de novato del Capítulo 3

⚠️ 1. Olvidar el dólar al nombrar la variable

Escribir `edad = 25;` en vez de `$edad = 25;`. PHP pensará que estáis llamando a una constante y lanzará un error de sintaxis al intentar asignarle un valor.

⚠️ 2. Nombres con guiones o espacios

Las variables no pueden tener espacios ni guiones medios: `$mi-variable` lo interpreta como la resta de `$mi` menos `variable`. Usad siempre la convención `camelCase`: `$miVariable`.

CONSOLIDACIÓN DE CONOCIMIENTOS

Comprobación de conocimientos del Capítulo 3

Cinco preguntas clave de aula para consolidar sintaxis y variables antes de los operadores

1. ¿Por qué se considera una mala práctica cerrar con ?> en un archivo que solo contiene código PHP?

> Porque cualquier espacio en blanco o salto de línea invisible tras el cierre se emite como salida al cliente, provocando el bloqueo del envío posterior de cabeceras («headers already sent»).

2. ¿Qué diferencia de comportamiento existe entre escribir una cadena con comillas simples frente a comillas dobles?

> Las comillas simples toman el texto de forma literal sin interpretar variables ni secuencias especiales, mientras que las comillas dobles interpolan automáticamente las variables contenidas en su interior.

3. ¿Qué hace la directiva declare(strict_types=1); al principio del archivo?

> Obliga al intérprete de PHP a verificar con exactitud los tipos de datos en parámetros y retornos de funciones, impidiendo conversiones automáticas imprevistas y lanzando excepciones de tipo TypeError ante discrepancias.

4. ¿Para qué se utiliza la constante mágica __DIR__ en la inclusión de ficheros?

> Devuelve la ruta absoluta del directorio donde reside el archivo actual en el disco duro, permitiendo construir rutas indestructibles independientes del directorio de trabajo.

5. ¿Por qué la instrucción echo resulta insuficiente para comprobar una variable booleana con valor false?

> Porque echo convierte internamente false en una cadena vacía («»), haciendo que no se dibuje nada en pantalla. Para ver el tipo bool(false) real en memoria es necesario utilizar var_dump().

Lo que nos espera en el Capítulo 4

¡Genial, alumnos! Ya dominamos la materia prima de PHP. En el **Capítulo 4** aprenderemos a operar con estos datos: **operadores aritméticos, el peligro mortal del doble igual (==) frente al triple igual (===), operadores lógicos y los operadores modernos de PHP 8 como la nave espacial (⇔) y la fusión de null (??)**.

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Sintaxis básica y etiquetas de PHP*: https://www.php.net/manual/es/language.basic-syntax.php
[2] *PHP Manual - Tipado estricto y declaraciones de tipo*: https://www.php.net/manual/es/language.types.declarations.php
[3] *PHP Manual - Constantes predefinidas y constantes mágicas*: https://www.php.net/manual/es/language.constants.magic.php
[4] *PHP Manual - Función var_dump() y depuración de variables*: https://www.php.net/manual/es/function.var-dump.php

CAPÍTULO 04 · CÁLCULOS Y LÓGICA

Operadores, expresiones y operaciones matemáticas

Aritmética, asignaciones combinadas, la batalla mortal == vs === y los nuevos operadores de PHP 8

> Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Hoy vamos a poner a trabajar a las variables. Guardar datos en cajas de memoria está muy bien, pero un programa cobra vida cuando empezamos a calcular precios, comparar respuestas y tomar decisiones automáticas.

En PHP una **expresión** es cualquier fragmento de código que produce un valor. Los **operadores** son los símbolos matemáticos y lógicos que combinan esos valores. Empecemos por los operadores aritméticos esenciales que utilizaréis a diario:

Operador	Nombre	Ejemplo en PHP	Resultado	Comentario didáctico
`+`	Suma	`$a = 10 + 5;`	`15`	Suma numérica. ¡Ojo! En PHP nunca se usa para unir textos.
`-`	Resta	`$a = 20 - 8;`	`12`	Diferencia entre dos operandos.
`*`	Multiplicación	`$a = 4 * 2.5;`	`10.0`	Producto. Si interviene un decimal, el resultado es float.
`/`	División	`$a = 10 / 4;`	`2.5`	División real. Si divides entre cero en PHP 8 lanza una excepción DivisionByZeroError.
`%`	Módulo (Resto)	`$a = 11 % 3;`	`2`	El resto de la división entera. Imprescindible para saber si un número es par (`$n % 2 === 0`).
`**`	Exponenciación	`$a = 2 ** 3;`	`8`	Dos elevado al cubo (2 * 2 * 2). Introducido en versiones modernas.

Asignación combinada: Código más corto y limpio

En vez de escribir `$total = $total + $envio;`, los programadores usamos atajos combinados: `$total += $envio;`. Y para texto, usamos `.=` para ir pegando frases.

LA REGLA DE ORO DEL AULA · SEGURIDAD

La batalla mortal: == (Débil) vs. === (Estricto)

Por qué en mis clases está terminantemente prohibido usar la igualdad débil

Fijaos bien en este apartado porque aquí se caen los alumnos en las entrevistas de trabajo. PHP tiene dos formas de comparar si dos cosas son iguales [1]:

`==` Igualdad débil (Conversión silenciosa)
Compara solo el valor aparente, obligando a uno de los datos a convertirse al tipo del otro antes de contrastar. Genera comportamientos inesperados absurdos.

`===` Igualdad estricta (Identidad absoluta)
Comprueba dos cosas a la vez: que tengan el mismo **tipo de dato** y el mismo **valor exacto**. Sin trucos ni conversiones ocultas en memoria.

La tabla de las trampas de la igualdad débil

Comparación	Con == (Débil)	Con === (Estricto)	Explicación del profesor
`0 == "0"`	`true`	`false`	Uno es int y el otro string. En estricto son tipos distintos.
`0 == ""`	`true` (en PHP 7)	`false`	Cero comparado con texto vacío. ¡Peligro mortal en logins!
`false == 0`	`true`	`false`	Un booleano jamás debería ser idéntico a un número entero.
`null == false`	`true`	`false`	La ausencia de dato (null) no es un valor booleano false.

⚠️ **Mandamiento docente: Usad siempre === y !==**
Grabaos esto a fuego: en PHP moderno usamos **siempre tres signos de igual** (`===`) para comparar igualdad y admiración con dos iguales (`!==`) para comprobar diferencia. El doble igual solo introduce agujeros de seguridad.

PHP 8 MODERNO · ELEGANCIA Y EFICIENCIA

Operadores modernos: Fusión de Null y Nave Espacial

Cómo ahorrar cientos de líneas de comprobaciones con ??, ?: y ⇔

PHP 8 ha incorporado operadores que hacen que nuestro código parezca magia limpia y legible:

operadores_modernos.php — Visual Studio Code

operadores_modernos.php

```php
<?php
declare(strict_types=1);

// 1. Operador Fusión de Null (??): Si $_GET['usuario'] no existe o es null, usa 'Invitado'
$usuario = $_GET['usuario'] ?? 'Invitado';

// 2. Operador Ternario (?:): Atajo elegante para un if/else simple
$esMayor = ($edad >= 18) ? 'Pase adelante' : 'Acceso denegado';

// 3. Operador Nave Espacial (<=>): Devuelve -1 si menor, 0 si igual, 1 si mayor
$comparacion = 15 <=> 20; // Devuelve -1 (ideal para funciones usort)
```

Operadores lógicos y tablas de verdad

Para combinar condiciones utilizamos `&&` (Y lógico, ambas deben cumplirse), `||` (O lógico, basta con que una se cumpla) y `!` (Negación). Fijaos en que nunca usamos `and` ni `or` en minúscula porque tienen una precedencia extraña que provoca errores silenciosos.

CONSOLIDACIÓN DE CONOCIMIENTOS

Comprobación de conocimientos del Capítulo 4

Cinco preguntas clave sobre cálculos y comparaciones antes de las estructuras condicionales

1. ¿Qué resultado produce la expresión 10 % 3 en PHP y para qué se suele utilizar?

> Produce 1 (el resto de la división entera 10 entre 3). Se utiliza habitualmente para determinar si un número es par o impar ($n % 2 === 0) o para alternar colores de fila.

2. ¿Por qué 0 === "0" devuelve false en PHP mientras que 0 == "0" devuelve true?

> Porque el operador estricto === exige que ambos operandos compartan el mismo tipo y valor. 0 es de tipo int y "0" es de tipo string, por lo que son tipos distintos.

3. ¿Qué ventaja aporta el operador de fusión de null (??) al leer parámetros como $_GET['id']?

> Comprueba de forma segura si la clave existe y no es nula. Si no existe, devuelve el valor por defecto indicado sin lanzar ninguna advertencia de tipo Undefined array key.

4. ¿Qué tres valores posibles puede devolver el operador nave espacial ($a ⇔ $b)?

> Devuelve -1 si $a es menor que $b, 0 si ambos son iguales, y 1 si $a es mayor que $b. Es el operador estándar para algoritmos de ordenación con usort().

5. ¿Por qué se desaconseja utilizar los operadores and y or en lugar de && y ||?

> Porque and y or tienen una precedencia inferior al operador de asignación (=), lo que provoca que $x = false or true asigne false a $x antes de evaluar la condición.

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Operadores de comparación y tablas de tipos*: https://www.php.net/manual/es/language.operators.comparison.php
[2] *PHP Manual - Operadores aritméticos y de asignación*: https://www.php.net/manual/es/language.operators.arithmetic.php
[3] *PHP Manual - Operador de fusión de null y operador ternario*: https://www.php.net/manual/es/language.operators.comparison.php#language.operators.comparison.coalesce

CAPÍTULO 05 · LÓGICA CONDICIONAL

Estructuras de control: if, else y elseif

La metáfora del portero de discoteca, bifurcaciones de código y anidamiento limpio

> Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Hasta ahora nuestros programas han sido muy obedientes: ejecutaban la línea 1, luego la 2 y luego la 3. Pero un programa inteligente tiene que saber tomar caminos distintos según lo que ocurra en el mundo real.

Para entender los condicionales, pensad en el **portero de una discoteca**. El portero tiene una lista de normas en la cabeza: si eres mayor de 18 y llevas calzado adecuado, pasas. Si no cumples la edad, a casa. En PHP esa toma de decisiones se construye con `if`, `else` y `elseif` [1]:

portero_demo.php — Visual Studio Code

portero_demo.php

```php
<?php
declare(strict_types=1);

$edad = 17;
$tieneEntradaVip = true;

if ($edad >= 18) {
    echo "<p>Acceso concedido a la sala general.</p>";
} elseif ($tieneEntradaVip === true) {
    echo "<p>Acceso especial a la zona juvenil VIP.</p>";
} else {
    echo "<p>Acceso denegado. Vuelve cuando cumplas 18.</p>";
}
```

El orden de evaluación importa muchísimo

PHP evalúa las condiciones de arriba hacia abajo. En cuanto encuentra la **primera que sea verdadera**, ejecuta ese bloque y se salta olímpicamente todas las demás. Si ponéis una condición general antes que una específica, la específica no se ejecutará jamás.

EVOLUCIÓN DEL LENGUAJE · SEGURIDAD

Switch clásico vs. La moderna expresión match

Por qué match de PHP 8 deja obsoleto a switch y elimina los errores por falta de break

Cuando tenemos que comparar una variable contra cinco o diez opciones posibles (por ejemplo, el rol de un usuario: admin, editor, alumno, invitado), encadenar muchos `elseif` se vuelve tedioso. Tradicionalmente usábamos `switch`, pero en PHP 8 ha nacido la joya de la corona: **match** [2].

Característica	Estructura switch clásica	Expresión match (PHP 8.x)
Tipo de comparación	Comparación débil (`==`). Peligro de conversiones silenciosas.	**Comparación estricta (===)** . Seguridad total.
Control de flujo	Requiere escribir `break;` en cada caso. Si lo olvidas, cae al siguiente.	No necesita `break` . Ejecuta un solo brazo y termina.
Retorno de valor	Es una estructura; no devuelve nada directamente a una variable.	**Es una expresión** . Se asigna directamente a una variable.

match_moderno.php — Visual Studio Code

match_moderno.php

```php
$rol = 'editor';

$permisos = match ($rol) {
    'admin' => 'Control total del sistema',
    'editor', 'redactor' => 'Puede crear y editar contenidos',
    'alumno' => 'Acceso a cursos y ejercicios',
    default => 'Acceso público de solo lectura',
};

```

SINTAXIS ALTERNATIVA · HTML LIMPIO

Condicionales en plantillas HTML y fallos típicos

Uso de if: ... endif; para no perderse entre llaves y el error clásico if ($a = 5)

Cuando mezclamos código PHP con etiquetas HTML en una página web real, llenar el archivo de llaves `{` y `}` es una fuente inagotable de despistes. Por eso PHP ofrece una **sintaxis alternativa con dos puntos y endif** [1]:

perfil_plantilla.php — Visual Studio Code

perfil_plantilla.php

```php
<?php if ($usuarioLogueado): ?>
    <div class="user-panel">
        <h3>Bienvenido, <?= $nombre ?></h3>
        <a href="/logout.php">Cerrar sesión</a>
    </div>
<?php else: ?>
    <a href="/login.php" class="btn">Iniciar Sesión</a>
<?php endif; ?>
```

⚠️ **El error clásico de novato: if ($x = 10)**

Escribir un solo igual dentro de la condición. Eso no compara: **asigna el valor 10 a la variable $x** y como 10 es distinto de cero, la condición siempre evalúa como verdadera. Comprobad siempre que ponéis triple igual: `if ($x === 10)`.

CONSOLIDACIÓN DEL APRENDIZAJE

Comprobación de conocimientos del Capítulo 5

Cinco preguntas clave de aula sobre condicionales, match y sintaxis para plantillas

1. ¿Qué ocurre si en una estructura if / elseif varias condiciones son verdaderas al mismo tiempo?

> Solo se ejecuta el bloque de código correspondiente a la primera condición verdadera encontrada. Las siguientes se ignoran por completo.

2. ¿Qué ventaja fundamental de seguridad tiene la expresión match de PHP 8 frente a switch?

> Match utiliza comparación estricta (===) en lugar de comparación débil (==), impidiendo que tipos incompatibles coincidan accidentalmente.

3. ¿Qué sucede si ninguna opción coincide en un match y no se ha definido el caso default?

> PHP lanza de inmediato un error fatal de tipo UnhandledMatchError, asegurando que ningún caso imprevisto pase desapercibido en el sistema.

4. ¿Por qué es recomendable utilizar if: ... endif; en lugar de llaves al mezclar PHP con HTML?

> Mejora drásticamente la legibilidad del documento HTML y evita confusiones sobre a qué etiqueta o condicional pertenece cada llave de cierre.

5. ¿Qué significa la etiqueta corta <?= $variable ?>?

> Es el equivalente exacto a escribir <?php echo $variable; ?>. Es el estándar limpio para imprimir datos en vistas de plantillas HTML.

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Estructuras de control: if, else, elseif y sintaxis alternativa*: https://www.php.net/manual/es/control-structures.if.php
[2] *PHP Manual - Expresión match (Novedad PHP 8.0)*: https://www.php.net/manual/es/control-structures.match.php

CAPÍTULO 06 · CONTROL DE FLUJO

Bucles y repetición controlada: while y do-while

La metáfora de fregar platos, condiciones de parada y cómo evitar colgar el servidor con un bucle infinito

> Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Los ordenadores son pésimos teniendo ideas originales, pero son insuperables haciendo tareas repetitivas millones de veces sin cansarse ni quejarse. Hoy vamos a domar los bucles en PHP.

Para entender el bucle `while`, pensad en la tarea de **fregar platos en la cocina** de nuestro restaurante: mientras haya platos sucios en la pila, coges un plato, lo enjabonas y lo aclaras. Cuando la pila esté vacía, paras y sales de la cocina [1].

while_demo.php — Visual Studio Code

while_demo.php

```php
<?php
$platosEnPila = 3;

while ($platosEnPila > 0) {
    echo "<p>Fregando plato. Quedan: {$platosEnPila}</p>";
    $platosEnPila--; // ¡Paso vital! Restamos uno en cada vuelta
}
echo "<p>¡Pila limpia! Cocina cerrada.</p>";
```

⚠️ El terror del bucle infinito: Olvidar actualizar la variable

Si olvidáis escribir `$platosEnPila--;`, la variable siempre valdrá 3, la condición `3 > 0` siempre será verdadera y el servidor web se quedará atrapado en un bucle eterno consumiendo el 100% de la CPU hasta que venza el tiempo límite de ejecución (`max_execution_time`). Comprobad siempre que la condición tiene una salida garantizada.

ESTRUCTURAS DE ITERACIÓN · CONTEO Y COLECCIONES

El bucle for y el rey indiscutible: foreach

Conteo con precisión matemática y recorrido de catálogos y colecciones de datos

Cuando sabéis de antemano cuántas veces queréis repetir una acción (por ejemplo, mostrar los números del 1 al 10 o dibujar las páginas de una paginación), usamos el bucle **for** [1]:

bucle_for.php — Visual Studio Code

bucle_for.php

```php
for ($i = 1; $i <= 5; $i++) {
    echo "<span class='page-num'>Página {$i}</span> ";
}
```

El rey indiscutible de la web: foreach

En el desarrollo web profesional, el 95% de los bucles que escribiréis serán `foreach`. Está diseñado específicamente para recorrer colecciones de datos y tablas de bases de datos sin preocuparos por índices ni contadores manuales:

carrito_demo.php — Visual Studio Code

carrito_demo.php

```php
$carrito = [
    'Teclado Mecánico' => 89.99,
    'Ratón Ergonómico' => 45.50,
    'Monitor 27 Pulgadas' => 210.00
];

foreach ($carrito as $producto => $precio) {
    echo "<li><strong>{$producto}</strong>: {$precio} €</li>";
}
```

CONTROL QUIRÚRGICO · OPTIMIZACIÓN

Alterando el rumbo: break y continue

Cómo abortar una búsqueda cuando encontramos el dato y cómo saltar a la siguiente vuelta

A veces necesitamos alterar la marcha natural de un bucle antes de que termine todas sus vueltas previstas [2]:

break **Interrupción inmediata**

Termina el bucle al instante y salta a la primera línea fuera de él. Ideal si buscáis un usuario en una lista de mil personas: cuando lo encontráis, ¿para qué seguir mirando las otras 999?

continue **Salto a la siguiente vuelta**

Abandona la vuelta actual y pasa de inmediato a evaluar la siguiente iteración. Muy útil para saltarse productos sin stock o usuarios dados de baja sin procesar el resto del bloque.

filtro_stock.php — Visual Studio Code

filtro_stock.php

```php
foreach ($productos as $item) {
    // Si el producto está agotado, nos lo saltamos y seguimos con el siguiente
    if ($item['stock'] === 0) {
        continue;
    }

    echo "<div class='item'>{$item['nombre']} - Disponible</div>";
}

```

CONSOLIDACIÓN DEL BLOQUE II

Comprobación de conocimientos del Capítulo 6

Cinco preguntas clave de aula sobre iteraciones, foreach y control de bucles

1. ¿Qué diferencia sustancial existe entre un bucle while y un bucle do-while?

> While comprueba la condición antes de ejecutar el bloque (si es falsa, no se ejecuta nunca). Do-while ejecuta el bloque al menos una vez antes de verificar la condición.

2. ¿Por qué el bucle foreach es la estructura preferida para recorrer arrays asociativos?

> Porque itera automáticamente por cada par clave-valor sin necesidad de gestionar contadores numéricos manuales ni preocuparse por los límites de la colección.

3. ¿Qué hace la sentencia continue dentro de una iteración?

> Detiene la ejecución del bloque actual de esa vuelta del bucle e inicia inmediatamente la comprobación de la siguiente iteración.

4. ¿Cómo se evita que un bucle while se convierta en un bucle infinito que bloquee el servidor?

> Asegurando que dentro del cuerpo del bucle exista al menos una instrucción que modifique el estado de las variables evaluadas en la condición de parada.

5. ¿Qué ocurre si utilizamos la sentencia break dentro de un bucle anidado?

> Por defecto interrumpe y sale del bucle más interno en el que se encuentra. Si se desea salir de varios niveles anidados, se puede especificar el número de niveles (ej. break 2;).

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Bucles: while, do-while, for y foreach*: https://www.php.net/manual/es/control-structures.while.php
[2] *PHP Manual - Sentencias de interrupción: break y continue*: https://www.php.net/manual/es/control-structures.break.php

CAPÍTULO 07 · COLECCIONES DE DATOS

Arrays al microscopio: El arma secreta de PHP

Arrays indexados, asociativos, matrices multidimensionales y el operador de desempaquetado

> Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Si las variables normales eran cajas donde solo cabe un dato a la vez, los **arrays** son como un archivador de oficina con cientos de compartimentos numerados o con etiquetas personalizadas. Sin ellos, no existiría el desarrollo web.

En otros lenguajes como C o Java existen diferencias rígidas entre vectores, listas, diccionarios y mapas. En PHP todo eso se unifica bajo una única estructura extremadamente flexible: el array [1].

1. Arrays indexados vs. Arrays asociativos

[0] Arrays indexados (Numéricos)

Las posiciones se numeran automáticamente empezando siempre por el **cero**: `$frutas = ['Manzana', 'Pera', 'Plátano'];`. `$frutas[0]` es 'Manzana'.

⇒ Arrays asociativos (Clave ⇒ Valor)

Cada dato se identifica con una palabra clave representativa: `$alumno = ['nombre' ⇒ 'Carlos', 'nota' ⇒ 8.5];`. Mucho más descriptivo y legible.

2. Añadir, modificar y eliminar elementos

arrays_basicos.php — Visual Studio Code

arrays_basicos.php

```php
$alumnos = ['Ana', 'David'];
$alumnos[] = 'Elena'; // Añade al final de la lista

$curso = ['aula' ⇒ 'A1', 'plazas' ⇒ 20];
$curso['profesor'] = 'Ramón Abramo'; // Añade una nueva clave
unset($curso['plazas']); // Elimina la clave y su valor


```

ESTRUCTURAS COMPLEJAS · EL MUNDO REAL

Arrays multidimensionales y la navaja suiza nativa

Tablas de bases de datos simuladas, array_map, in_array y el operador spread (...)

En una tienda online real no tenemos una sola variable suelta: tenemos una tabla de productos donde cada producto tiene su nombre, precio y stock. Eso es exactamente un **array multidimensional** (un array que contiene otros arrays dentro) [1]:

catalogo_multi.php — Visual Studio Code

catalogo_multi.php

```php
$catalogo = [
    ['id' => 1, 'nombre' => 'Portátil Pro', 'precio' => 899.00],
    ['id' => 2, 'nombre' => 'Monitor 4K', 'precio' => 299.50]
];

// Para acceder al nombre del primer producto:
echo $catalogo[0]['nombre']; // Imprime: Portátil Pro
```

Funciones nativas imprescindibles para arrays

Función	Para qué sirve	Ejemplo en código
`count($arr)`	Devuelve el número total de elementos.	`$total = count($alumnos);`
`in_array($val, $arr)`	Comprueba si un valor existe en el array.	`if (in_array('admin', $roles))`
`array_key_exists($k, $arr)`	Verifica si una clave asociativa existe.	`if (array_key_exists('email', $datos))`
`array_merge($a, $b)`	Fusiona dos arrays en uno solo.	`$todos = array_merge($locales, $remotos);`
`array_map($fn, $arr)`	Aplica una función a cada elemento y devuelve el array transformado.	`$preciosConIva = array_map(fn($p) => $p * 1.21, $precios);`

El operador de desempaquetado (...) de PHP 8

Podéis unir arrays de forma moderna sin llamar a `array_merge()`: `$combinado = [...$frutas, ...$verduras];`. Es mucho más rápido y elegante.

CONSOLIDACIÓN DE CONOCIMIENTOS

Comprobación de conocimientos del Capítulo 7

Cinco preguntas clave de aula sobre arrays, claves asociativas y transformaciones

1. ¿Cuál es el índice del primer elemento en un array indexado numéricamente en PHP?

El índice 0. Intentar acceder a la posición 1 creyendo que es el primer elemento es un error típico que descarta el primer dato de la lista.

2. ¿Qué ocurre si intentamos leer una clave que no existe dentro de un array asociativo?

PHP lanza una advertencia de tipo Warning: Undefined array key. Se evita comprobando previamente con isset() o utilizando el operador de fusión de null (??).

3. ¿Qué diferencia sustancial existe entre la función sort() y la función asort()?

sort() ordena los valores y reasigna los índices numéricos desde cero, mientras que asort() ordena los valores pero conserva intactas las claves asociativas originales.

4. ¿Cómo elimina la función unset() un elemento de un array?

Elimina la clave y el valor especificado de la memoria, pero no reordena los índices numéricos restantes (quedan huecos en la numeración).

5. ¿Qué ventaja ofrece array_filter() frente a un bucle foreach tradicional?

Permite filtrar colecciones de datos en una sola línea mediante una función de retorno booleano (callback), produciendo un código más conciso y declarativo.

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Tipos compuestos: Arrays y funciones de manipulación*: https://www.php.net/manual/es/language.types.array.php
[2] *PHP Manual - Funciones de arrays (array_map, in_array, usort)*: https://www.php.net/manual/es/ref.array.php

CAPÍTULO 08 · TRATAMIENTO DE TEXTO

Manipulación y formateo de texto avanzado

Búsquedas modernas, la frontera explode/implode y el soporte multibyte (mb_*) para tildes y eñes

Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Los humanos nos comunicamos mediante palabras, pero para un ordenador un texto no es más que una ristra de bytes en memoria. Hoy vamos a aprender a buscar, recortar, transformar y validar texto como auténticos artesanos del código.

1. Las funciones modernas de búsqueda en PHP 8

Históricamente en PHP teníamos que usar `strpos()` y pelearnos con el triple igual (`=== false`) para saber si una palabra estaba dentro de otra. En PHP 8 llegaron las funciones más esperadas [1]:

str_contains **¿Contiene el texto?** `str_contains($email, '@')` devuelve directamente `true` o `false`. Limpio y sin rodeos.

starts / ends **¿Empieza o termina por?** `str_starts_with($url, 'https://')` y `str_ends_with($archivo, '.pdf')`. Claridad meridiana.

2. El puente entre cadenas y arrays: explode() e implode()

¿Qué hacéis cuando un usuario introduce una lista de etiquetas separadas por comas (`"php, laragon, mysql"`)? La función `explode()` la convierte en un array. Y su compañera `implode()` hace justo lo contrario:

explode_demo.php — Visual Studio Code

explode_demo.php

```php
$tagsTexto = "php, laragon, mysql, apache";
$tagsArray = explode(", ", $tagsTexto); // Array de 4 elementos

// Unimos de nuevo los elementos con guiones:
$slug = implode(" - ", $tagsArray); // "php - laragon - mysql - apache"

```

CODIFICACIÓN Y VALIDACIONES · SIN DOLOR

El lío de los caracteres especiales y Regex sin pánico

Por qué strlen falla con tildes y cómo usar preg_match para validar DNI, emails y teléfonos

El peligro de strlen() con caracteres en español

Si escribís `strlen("Ramón")`, PHP os dirá que mide **6 caracteres** en vez de 5. ¿Por qué? Porque la función clásica `strlen()` cuenta bytes, y en codificación UTF-8 la letra con tilde `ó` ocupa 2 bytes. En mis clases usamos siempre las funciones multibyte: `mb_strlen("Ramón", "UTF-8")` devuelve 5 exactos [1].

Expresiones regulares (Regex) para humanos

Una expresión regular es un patrón que describe cómo debe ser un texto. Con la función `preg_match()` podemos validar entradas de formularios de forma infalible [2]:

validar_regex.php — Visual Studio Code

validar_regex.php

```php
$telefono = "612345678";
// Patrón: empieza por 6, 7, 8 o 9 y le siguen 8 dígitos numéricos exactos
$patron = '/^[6-9][0-9]{8}$/';

if (preg_match($patron, $telefono)) {
    echo "<p style='color:green;'>Teléfono español válido.</p>";
}
```

Para emails: ¡No inventéis expresiones regulares!

Escribir una expresión regular perfecta para un correo electrónico es dificilísimo. Para validar emails, PHP incluye la función nativa oficial: `filter_var($email, FILTER_VALIDATE_EMAIL)`. Rápido, seguro y sin fallos.

CONSOLIDACIÓN DE CONOCIMIENTOS

Comprobación de conocimientos del Capítulo 8

Cinco preguntas clave de aula sobre funciones de texto, multibyte y validaciones

1. ¿Qué función de PHP 8 permite comprobar si un texto contiene una subcadena sin recurrir a strpos()?

La función str_contains($texto, $subcadena), que devuelve directamente un valor booleano true o false.

2. ¿Por qué se debe utilizar mb_substr() en lugar de substr() al trabajar con textos en español?

Porque substr() corta por conteo de bytes puros. Si corta justo en medio de un carácter multibyte (como una tilde o una eñe), genera un carácter corrupto o inválido.

3. ¿Qué operación realiza la función explode(',', $linea)?

Divide la cadena $linea utilizando la coma como delimitador y devuelve un array con cada uno de los fragmentos resultantes.

4. ¿Qué significa el modificador 'i' al final de un patrón de expresión regular como '/php/i'?

Indica que la búsqueda debe ser insensible a mayúsculas y minúsculas (case-insensitive), coincidiendo con 'PHP', 'php' o 'Php'.

5. ¿Cuál es el método nativo recomendado por PHP para validar una dirección de correo electrónico?

Utilizar filter_var($email, FILTER_VALIDATE_EMAIL), que implementa los estándares RFC oficiales de formato de correo.

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Funciones de manipulación de cadenas (str_contains, explode)*: https://www.php.net/manual/es/ref.strings.php
[2] *PHP Manual - Expresiones regulares compatibles con Perl (PCRE / preg_match)*: https://www.php.net/manual/es/ref.pcre.php

CAPÍTULO 09 · MODULARIDAD Y CÓDIGO LIMPIO

Funciones: Escribe una vez y utilízalo mil

Parámetros, la diferencia radical echo vs return, tipado estricto y argumentos con nombre

Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Si os encontráis escribiendo el mismo trozo de código tres veces en distintas partes de una aplicación, parad de inmediato: estáis violando el principio sagrado del desarrollo de software: **DRY** (*Don't Repeat Yourself*, No te repitas). Hoy vamos a encapsular la lógica en funciones profesionales.

1. La gran frontera: echo frente a return

Muchos alumnos confunden mostrar algo en pantalla con devolver un valor. Una función no debe ser una impresora; una función debe ser una calculadora que procesa datos y os **devuelve** el resultado para que vosotros decidáis qué hacer con él [1]:

funciones_demo.php — Visual Studio Code

funciones_demo.php

```php
<?php
declare(strict_types=1);

// Función con tipado en argumentos y tipo de retorno float:
function calcularIva(float $importe, float $tasa = 0.21): float {
    return $importe * $tasa;
}

$cuotaIva = calcularIva(100.0); // Devuelve 21.0 a la variable
echo "<p>El IVA es: {$cuotaIva} €</p>";
```

2. Argumentos con nombre (Named Arguments) de PHP 8

En PHP 8 podéis pasar los parámetros indicando su nombre explícito: `calcularIva(tasa: 0.10, importe: 50.0);`. Esto hace que el orden de los argumentos dé igual y que el código se entienda con solo leerlo [1].

ARQUITECTURA MODULAR · ORGANIZACIÓN

Organización en módulos: require_once y __DIR__

Diferencias entre include, require, include_once y require_once sin romper la web

Nadie escribe una aplicación web entera en un solo archivo de diez mil líneas. Dividimos el proyecto en piezas especializadas: cabeceras, pies de página, funciones y configuraciones. Para ensamblar las piezas usamos las sentencias de inclusión [2]:

Sentencia	Qué hace si el archivo no existe	Control de duplicados	Veredicto docente
`include`	Lanza un Warning y **continúa ejecutando** el script.	No comprueba si ya estaba incluido.	Peligroso si faltan funciones vitales.
`require`	Lanza un Fatal Error y **detiene el programa en seco**.	No comprueba duplicados.	Bueno para archivos indispensables.
`include_once`	Lanza Warning y continúa.	Si ya se incluyó antes, lo omite.	Rara vez justificado.
`require_once`	Lanza Fatal Error y detiene el script.	**Garantiza una sola inclusión**.	**El estándar profesional** para funciones y clases.

La regla indestructible de las rutas

index.php — Visual Studio Code

index.php

```php
// Siempre construimos la ruta absoluta usando la constante mágica __DIR__:
require_once __DIR__ . '/includes/funciones.php';
require_once __DIR__ . '/includes/config.php';
```

⚠️ **Por qué require_once evita el error «Cannot redeclare»**

Si utilizáis `require` normal y por despiste incluís el archivo de funciones dos veces, PHP intentará declarar la misma función dos veces y reventará con un error fatal: *«Cannot redeclare function»*. Usando `require_once`, PHP sabe que ya está cargado y no vuelve a tocarlo.

CONSOLIDACIÓN DEL BLOQUE III

Comprobación de conocimientos del Capítulo 9

Cinco preguntas clave de aula sobre funciones, tipos de retorno y modularidad

1. ¿Qué diferencia práctica existe entre una función que hace echo y una que hace return?

> echo envía el texto directamente a la salida del navegador sin permitir que el programa reutilice el dato. return devuelve el valor producido al punto de llamada para que pueda guardarse en una variable o usarse en otros cálculos.

2. ¿Qué significa el tipo de retorno void en la cabecera de una función?

> Indica de forma explícita que la función ejecuta una acción pero no devuelve ningún valor útil mediante return.

3. ¿Qué ocurre si pasamos un valor string a una función que declara un parámetro int con strict_types activo?

> PHP lanza de inmediato una excepción fatal de tipo TypeError, impidiendo conversiones automáticas imprevistas.

4. ¿Qué ventaja sustancial aporta require_once frente a require común?

> Comprueba si el archivo ya ha sido incorporado previamente en la ejecución del script. Si ya está cargado, no vuelve a procesarlo, evitando errores de redeclaración de funciones y clases.

5. ¿Cómo se define una función flecha (arrow function) concisa en PHP moderno?

> Utilizando la sintaxis fn($x) ⇒ $x * 2. Captura automáticamente las variables del ámbito exterior por valor y devuelve directamente el resultado de la expresión.

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Funciones definidas por el usuario, parámetros y retorno*: https://www.php.net/manual/es/functions.user-defined.php
[2] *PHP Manual - Inclusión de archivos: require_once, include_once*: https://www.php.net/manual/es/function.require-once.php

CAPÍTULO 10 · LA WEB EN ACCIÓN

Formularios web: La puerta de entrada del usuario

Superglobales $_GET y $_POST, saneamiento contra XSS y validación rigurosa de campos

Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. Hasta ahora nuestros programas han sido cajas cerradas donde nosotros mismos definíamos las variables en el código. Pero la web es interactiva: los usuarios introducen nombres, correos, seleccionan opciones y pulsan botones. Hoy aprenderemos a recibir esa información en PHP sin que nos cuelen un ataque.

1. Las variables superglobales: $_GET y $_POST

PHP crea automáticamente una serie de arrays asociativos especiales llamados **superglobales** [1]. Están disponibles en cualquier rincón del código sin necesidad de usar la palabra clave `global`:

$_GET **Peticiones por URL**

Recoge los parámetros que viajan visibles tras la interrogación: `tienda.php?categoria=calzado&orden=precio`. Ideal para filtros, páginas y búsquedas que se puedan compartir por enlace.

$_POST **Peticiones por Formulario**

Recoge los datos enviados de forma oculta en el cuerpo de la petición. Obligatorio para logins, registros, pagos y cualquier acción que modifique datos en el servidor.

2. La regla sagrada: «Nunca te fíes de los datos del usuario»

Todo lo que viene de un formulario es potencialmente hostil. Un usuario malicioso puede alterar el HTML del navegador, inyectar etiquetas `<script>` o meter textos de tres millones de caracteres. Para defendernos, aplicamos dos pasos indispensables:

1. **Filtrar y Validar**
   Comprobar que el dato cumple el tipo y formato esperado usando `filter_var()` o expresiones regulares.
2. **Sanear en salida (Escapar)**
   Pasar siempre cualquier texto de usuario por `htmlspecialchars($texto, ENT_QUOTES, 'UTF-8')` antes de imprimirlo en el HTML para neutralizar ataques XSS.

PATRÓN PROFESIONAL · VALIDACIÓN COMPLETA

Procesamiento seguro de un formulario de contacto

Detección del método POST, comprobación de errores y persistencia de valores en el formulario

Vamos a escribir el patrón clásico de formulario auto-procesado en un único archivo. Si hay errores, mostramos los avisos en rojo y conservamos lo que el usuario ya había escrito para que no tenga que rellenarlo todo otra vez:

contacto_seguro.php — Visual Studio Code

contacto_seguro.php

```php
<?php
$errores = [];
$nombre = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nombre = trim($_POST['nombre'] ?? '');
    $email = filter_var($_POST['email'] ?? '', FILTER_VALIDATE_EMAIL);

    if ($nombre === '') {
        $errores[] = 'El nombre es obligatorio.';
    }
    if ($email === false) {
        $errores[] = 'El correo electrónico no tiene un formato válido.';
    }
}
```

Comprobar siempre $_SERVER['REQUEST_METHOD']

Nunca uséis `if (isset($_POST['enviar']))` porque si el usuario envía el formulario pulsando la tecla Intro en lugar de hacer clic en el botón, el parámetro del botón no viaja y el código no se procesa. Comprobad siempre el método HTTP de la petición.

SUBIDA DE ARCHIVOS · ALMACENAMIENTO SEGURO

Subida de archivos con $_FILES al descubierto

El atributo enctype multipart, validación de tipos MIME y almacenamiento con move_uploaded_file

Para subir fotos, justificantes o PDFs, el formulario HTML debe llevar obligatoriamente el atributo `enctype="multipart/form-data"`. Cuando se envía, los datos del archivo no van a `$_POST`, sino al array bidimensional **$_FILES** [2]:

Clave en $_FILES['foto']	Qué información contiene	Para qué la usamos los programadores
`['name']`	Nombre original del archivo en el PC del usuario.	Para extraer la extensión (ej. `.jpg`). **¡Nunca guardar en disco con este nombre!**
`['tmp_name']`	Ruta temporal donde el servidor ha guardado el archivo.	El archivo físico que moveremos a la carpeta definitiva de subidas.
`['size']`	Tamaño en bytes del archivo.	Comprobar que no supera el límite permitido (ej. máx 2 MB).
`['error']`	Código de estado de la subida (0 = éxito).	Si vale `UPLOAD_ERR_OK` (0), la subida temporal se realizó con éxito.

subida_segura.php — Visual Studio Code

subida_segura.php

```php
// Generamos un nombre criptográficamente único para evitar sobreescrituras:
$nombreFinal = bin2hex(random_bytes(16)) . '.jpg';
$destino = __DIR__ . '/uploads/' . $nombreFinal;

if (move_uploaded_file($_FILES['foto']['tmp_name'], $destino)) {
    echo "<p>Archivo subido y protegido con éxito.</p>";
}
```

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Variables superglobales ($_POST, $_GET, $_FILES)*: https://www.php.net/manual/es/language.variables.superglobals.php
[2] *PHP Manual - Gestión de subidas de ficheros con POST*: https://www.php.net/manual/es/features.file-upload.post-method.php

CAPÍTULO 11 · ESTADO Y PERSISTENCIA

Estado en una web sin memoria: Cookies y Sesiones

La metáfora de la taquilla del gimnasio, banderas de seguridad y el ciclo de vida de $_SESSION

Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. El protocolo HTTP es completamente **desmemoriado (stateless)**. Cada vez que hacéis clic en un enlace, para el servidor sois un perfecto desconocido, aunque hayáis entrado hace dos segundos. Hoy vamos a dotar de memoria a nuestras webs.

La analogía de la taquilla del gimnasio

Para entender las sesiones, pensad en la recepción de un gimnasio: llegáis por la mañana, os identificáis con vuestro DNI (usuario y contraseña) y el recepcionista guarda vuestras pertenencias en una taquilla con candado dentro del vestuario (el **servidor**). A vosotros no os da la ropa: os da una pulsera con un número grabado (el **identificador de sesión**, `PHPSESSID`). Cada vez que pasáis por la puerta, mostráis la pulsera y el recepcionista sabe exactamente quiénes sois.

Cookie **En el navegador del usuario**
Pequeño archivo de texto que el servidor ordena guardar al cliente. Se usa para preferencias (ej. tema oscuro o idioma) y para guardar la pulsera con el ID de sesión.

Sesión **En el servidor con $_SESSION**
Los datos reales residen de forma segura en el disco del servidor. El usuario solo guarda la clave numérica, impidiendo que manipule su rol o saldo.

Creación segura de cookies con setcookie()

cookie_segura.php — Visual Studio Code

cookie_segura.php

```php
// Definición moderna con array de opciones seguras en PHP 8:
setcookie('tema', 'oscuro', [
    'expires' => time() + (86400 * 30), // 30 días
    'path' => '/',
    'secure' => true,     // Solo viaja por HTTPS
    'httponly' => true,   // ¡Vital! JavaScript no puede leerla (anti-robo XSS)
    'samesite' => 'Strict'
]);
```

TALLER PRÁCTICO · CONTROL DE ACCESO

Sistema de login y protección de páginas privadas

session_start(), control de credenciales, protección de rutas y cierre de sesión seguro

Para crear un área privada en nuestra web necesitamos tres piezas: el inicio de sesión, el guardián de las páginas protegidas y el botón de desconexión [1]:

login_proceso.php — Visual Studio Code

login_proceso.php

```php
<?php
session_start(); // Siempre la primera instrucción

// Si las credenciales coinciden, creamos la sesión en el servidor:
session_regenerate_id(true); // Regenera el ID para evitar Session Fixation
$_SESSION['autenticado'] = true;
$_SESSION['usuario'] = 'Carlos';
$_SESSION['rol'] = 'alumno';

header('Location: /panel_privado.php');
exit;
```

El guardián en panel_privado.php

panel_privado.php — Visual Studio Code

panel_privado.php

```php
<?php
session_start();

if (empty($_SESSION['autenticado'])) {
    header('Location: /login.php?error=no_autorizado');
    exit; // ¡Indispensable! Detiene la ejecución para que no vean la página
}
```

⚠ **Olvidar el exit tras un header('Location: ...')**

Muchos novatos escriben la redirección y se olvidan de poner `exit;` debajo. Aunque el navegador reciba la orden de redirigir, **PHP continúa ejecutando todo el código que haya debajo** en el servidor. Si abajo tenéis órdenes de borrado o datos confidenciales, se ejecutarán igualmente. Poned siempre `exit;`.

CONSOLIDACIÓN DE CONOCIMIENTOS

Comprobación de conocimientos del Capítulo 11

Cinco preguntas clave de aula sobre sesiones, cookies seguras y control de usuarios

1. ¿Dónde se almacenan físicamente los datos de $_SESSION y dónde reside la cookie PHPSESSID?

> Los datos de $_SESSION se guardan en el servidor (en disco o memoria temporal). La cookie PHPSESSID se almacena en el navegador del usuario y solo contiene el identificador numérico.

2. ¿Qué vulnerabilidad previene el parámetro 'httponly' ⇒ true al crear una cookie?

> Impide que scripts maliciosos de JavaScript (ataques XSS en el navegador) puedan leer el contenido de la cookie mediante document.cookie.

3. ¿Por qué es fundamental ejecutar session_regenerate_id(true) justo tras autenticar al usuario?

> Previene ataques de fijación de sesión (Session Fixation), invalidando el ID de sesión anterior y generando uno nuevo para el usuario autenticado.

4. ¿Cuáles son los tres pasos para destruir una sesión de forma 100% limpia en PHP?

> 1) Vaciar el array con $_SESSION = []; 2) Borrar la cookie de sesión del navegador con setcookie; 3) Destruir los datos en el servidor con session_destroy().

5. ¿Por qué la función session_start() debe llamarse antes de cualquier salida HTML?

> Porque session_start() necesita enviar la cabecera HTTP Set-Cookie al cliente. Si ya se ha emitido HTML, las cabeceras estarán bloqueadas («headers already sent»).

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *PHP Manual - Manejo de sesiones y función session_start()*: https://www.php.net/manual/es/book.session.php
[2] *PHP Manual - Función setcookie() y parámetros de seguridad*: https://www.php.net/manual/es/function.setcookie.php

CAPÍTULO 12 · BLINDAJE PROFESIONAL

Seguridad web en PHP: Blindando el servidor

Inyección SQL, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF) y hash de contraseñas

Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente. En la informática moderna no existen las páginas «pequeñas» que no interesan a nadie: en cuanto subáis una web a Internet, robots automatizados empezarán a escanear vuestros formularios buscando fallos cada minuto. Hoy vamos a cerrar todas las puertas a cal y canto.

1. Inyección SQL: El suicidio de concatenar cadenas

Ocurre cuando un programador ingenuo mete directamente una variable del usuario dentro de una consulta SQL: `"SELECT * FROM usuarios WHERE email = '$email'"`. Si un atacante escribe en el campo de texto `' OR '1'='1`, altera la lógica de la base de datos y entra al sistema sin contraseña [1].

La solución definitiva: Consultas preparadas con PDO

Jamás concatenamos variables en SQL. Usamos **marcadores preparados** (`:email`). El motor de base de datos trata el dato como un simple texto inofensivo, neutralizando el 100% de los intentos de inyección SQL. Lo veremos en profundidad en el Bloque V.

2. Contraseñas seguras: Por qué md5() y sha1() están prohibidos

En clase todavía veo a alumnos viejos usando `md5($password)`. **¡Ni se os ocurra!** El algoritmo MD5 se rompe en milisegundos mediante tablas arcoíris. Desde PHP 7 y 8, el estándar normativo es la función nativa **password_hash()** con algoritmo Bcrypt o Argon2ID [2]:

hash_demo.php — Visual Studio Code

hash_demo.php

```php
// 1. Al registrar al usuario: cifra con salt criptográfico automático
$hashGuardado = password_hash($clavePlana, PASSWORD_DEFAULT);

// 2. Al iniciar sesión: comprueba la contraseña contra el hash de la BD
if (password_verify($claveIntroducida, $hashGuardado)) {
    echo "<p>Contraseña correcta. Acceso autorizado.</p>";
}
```

DEFENSA ACTIVA · ATAQUES CSRF

Ataques CSRF y tokens antifalsificación

Cómo evitar que una web maliciosa ejecute acciones en vuestro nombre sin que os deis cuenta

El ataque **CSRF** (*Cross-Site Request Forgery*) consiste en engañar a la víctima para que pulse un enlace en una web trampa mientras tiene su sesión del banco o de la tienda abierta. El navegador envía la cookie automáticamente y el servidor cree que la petición es legítima [1].

El antídoto infalible: Tokens CSRF de un solo uso

En cada formulario generamos un código aleatorio secreto que guardamos en la sesión y en un campo oculto del formulario. Al recibir la petición, comprobamos que ambos coincidan mediante `hash_equals()`:

token_csrf.php — Visual Studio Code

token_csrf.php

```php
// Generar token al pintar el formulario:
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

// Al procesar el POST: validación estricta
if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
    die("Petición falsificada no autorizada.");
}
```

FUENTES Y DOCUMENTACIÓN OFICIAL

[1] *OWASP Top 10 - Vulnerabilidades web críticas (SQLi, XSS, CSRF)*: https://owasp.org/www-project-top-ten/
[2] *PHP Manual - Criptografía y hash seguro de contraseñas (password_hash)*: https://www.php.net/manual/es/book.password.php

CAPÍTULO 13 · BLOQUE V: BASES DE DATOS

El almacén de datos: Bases de datos relacionales y SQL

Tablas, claves primarias, relaciones y las cuatro consultas sagradas que mueven el mundo

¡Bienvenidos a la despensa del restaurante! En el capítulo 1 os conté que PHP es el cocinero y la base de datos es el almacén donde se guardan los ingredientes. Si apagáis el servidor, la memoria RAM de PHP se borra por completo, pero lo que guardéis en MariaDB o MySQL permanece intacto. Vamos a entender cómo hablarle a la despensa usando SQL.

1. ¿Qué es una base de datos relacional y cómo se organiza?

Imaginad una hoja de cálculo gigante de Excel, pero con esteroides y diseñada para que miles de personas lean y escriban a la vez sin corromperse. En una base de datos relacional (RDBMS como MariaDB o MySQL), la información se organiza en **tablas**:

Filas **Registros (Tuplas)**
Cada fila representa un elemento individual único: un alumno concreto, una factura emitida o un producto de la tienda.

Columnas **Campos (Atributos)**
Cada columna define un dato tipado: nombre (VARCHAR), edad (INT), precio (DECIMAL) o fecha_alta (DATETIME).

PK **Clave Primaria (Primary Key)**
El DNI del registro. Un número entero único, generalmente `AUTO_INCREMENT`, que identifica inequívocamente a cada fila.

FK **Clave Foránea (Foreign Key)**
El puente relacional. Un campo que apunta a la clave primaria de otra tabla (ej: en la tabla pedidos guardamos el `cliente_id`).

2. Las cuatro operaciones del lenguaje SQL (CRUD)

SQL (Structured Query Language) es el idioma universal de las bases de datos. No importa si usáis MariaDB, PostgreSQL u Oracle; las cuatro sentencias básicas son exactamente las mismas:

consultas_basicas.sql - Visual Studio Code

SQL consultas_basicas.sql

```sql
-- 1. CONSULTAR (READ): Alumnos de Santander ordenados por apellido
SELECT id, nombre, apellidos, email FROM alumnos
WHERE ciudad = 'Santander' ORDER BY apellidos ASC LIMIT 10;

-- 2. INSERTAR (CREATE): Dar de alta un nuevo curso
INSERT INTO cursos (titulo, horas, precio) VALUES ('PHP 8 Moderno', 80, 250.00);

-- 3. ACTUALIZAR (UPDATE): Modificar el email de un alumno específico
UPDATE alumnos SET email = 'nuevo@alpe.es' WHERE id = 42;

-- 4. ELIMINAR (DELETE): Borrar un registro por su clave primaria
DELETE FROM alumnos WHERE id = 42;
```

¡ATENCIÓN! La pesadilla del UPDATE y DELETE sin WHERE

En mis más de 25 años en el aula, he visto a alumnos sudar frío al ejecutar `UPDATE alumnos SET activo = 0;` o `DELETE FROM alumnos;` sin la cláusula `WHERE`. Si hacéis eso, modificaréis o borraréis **todos los registros de la base de datos de golpe**. ¡Tatuaros esto: jamás ejecutéis un UPDATE o DELETE sin comprobar dos veces la cláusula WHERE!

CAPÍTULO 13 · CONSULTAS AVANZADAS

Relacionando tablas: INNER JOIN y LEFT JOIN

Cómo combinar información de diferentes despensas en una única respuesta coherente

3. Relaciones entre tablas: No dupliquéis información

Una de las reglas sagradas del diseño de bases de datos relacionales es la **normalización**: jamás guardéis los datos completos del cliente dentro de cada fila de pedido. Si el cliente cambia de dirección, tendríais que actualizar 500 pedidos. En su lugar, separamos en tabla `clientes` y tabla `pedidos`, y las unimos mediante el `cliente_id`.

joins_relaciones.sql - Visual Studio Code

SQL joins_relaciones.sql

```sql
-- INNER JOIN: Devuelve solo las filas que coinciden en AMBAS tablas
SELECT p.id AS pedido_num, p.fecha, c.nombre, c.email, p.total
FROM pedidos p
INNER JOIN clientes c ON p.cliente_id = c.id
WHERE p.estado = 'completado';

-- LEFT JOIN: Devuelve TODOS los clientes, tengan o no pedidos asociados
SELECT c.nombre, c.telefono, p.id AS num_pedido
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
WHERE p.id IS NULL; -- ¡Clientes que nunca han comprado nada!
```

4. Herramientas de gestión visual: phpMyAdmin y HeidiSQL

Aunque un buen programador domina la consola y la sintaxis SQL, para diseñar tablas, ver datos rápidamente y exportar copias de seguridad usamos clientes gráficos:

Herramienta	Tipo	Dónde viene	Ventajas para el alumno
phpMyAdmin	Web (PHP)	Integrado en Laragon y XAMPP	Acceso inmediato desde el navegador en `http://localhost/phpmyadmin`. Excelente para importar/exportar archivos .sql.
HeidiSQL	Escritorio nativo	Incluido por defecto en Laragon	Ulparápido, autocompletado inteligente de tablas y columnas, visor de claves foráneas y diseñador visual de esquemas.
DBeaver	Multiplataforma	Instalación independiente	Permite conectar MariaDB, PostgreSQL, SQLite y Oracle en una sola ventana unificada. Ideal si trabajáis en macOS o Linux.

Pregunta de examen de Ramón: ¿Por qué usamos utf8mb4 y no utf8 simple en nuestras tablas?

> **Respuesta docente:** En MySQL, el juego de caracteres histórico `utf8` solo reservaba 3 bytes por carácter, lo que rompía la aplicación si un usuario introducía un emoji o ciertos caracteres asiáticos especiales. El estándar moderno oficial es siempre **utf8mb4** con cotejamiento **utf8mb4_unicode_ci** (o `utf8mb4_0900_ai_ci`), que soporta el catálogo Unicode completo de 4 bytes.

CAPÍTULO 13 · TALLER PRÁCTICO

Laboratorio SQL: Diseño del esquema de una academia

Creación de tablas con claves foráneas e integridad referencial paso a paso

5. Script de creación de base de datos para nuestras prácticas

Abrid HeidiSQL o phpMyAdmin y ejecutad este script SQL completo. Esta será la base de datos que conectaremos desde PHP en el siguiente capítulo:

academia_alpe.sql - Visual Studio Code

SQL academia_alpe.sql

```sql
CREATE DATABASE IF NOT EXISTS academia_alpe
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE academia_alpe;

CREATE TABLE cursos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(120) NOT NULL,
  precio DECIMAL(8,2) NOT NULL DEFAULT 0.00
) ENGINE=InnoDB;

CREATE TABLE alumnos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  curso_id INT NOT NULL,
  CONSTRAINT fk_alumno_curso FOREIGN KEY (curso_id) REFERENCES cursos(id)
) ENGINE=InnoDB;
```

6. Ejercicios propuestos por Ramón

1. **Población de datos de prueba**
   Insertad mediante sentencias `INSERT` 3 cursos distintos ('PHP 8', 'JavaScript', 'Bases de Datos') y 5 alumnos matriculados en ellos.

2. **Consulta combinada con INNER JOIN**
   Escribid una consulta SQL que devuelva una tabla con: el nombre del alumno, su email y el título del curso en el que está matriculado.

3. **Comprobación de integridad referencial**
   Intentad insertar un alumno con un `curso_id = 999` inexistente. Comprobad cómo la base de datos os rechaza la consulta gracias a la clave foránea.

Fuentes y Documentación Oficial

Documentación oficial de MariaDB SQL: *mariadb.com/kb/en/sql-statements/*.
Manual de referencia de MySQL 8.0: *dev.mysql.com/doc/refman/8.0/en/*.

CAPÍTULO 14 · BLOQUE V: BASES DE DATOS

Conectando PHP con la Base de Datos: PDO

Conexión robusta mediante DSN, modo de errores con excepciones y sentencias preparadas

¿Os acordáis de los tutoriales de los años 2000 que usaban `mysql_connect()` o `mysql_query()`? Esas funciones están **muertas y enterradas** desde PHP 7.0 por inseguras y obsoletas. En este curso vamos a usar la herramienta estándar de la industria: **PDO (PHP Data Objects)**, orientada a objetos, segura y compatible con múltiples motores de base de datos.

1. El DSN y la conexión infalible con try / catch

Para conectar PHP a MariaDB o MySQL necesitamos definir el DSN (Data Source Name), indicando el motor, el host, el nombre de la base de datos y la codificación de caracteres:

conexion.php - Visual Studio Code

PHP conexion.php

```php
<?php
declare(strict_types=1);

$host    = 'localhost';
$db      = 'academia_alpe';
$user    = 'root';
$pass    = ''; // Por defecto vacío en Laragon / XAMPP
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
$opciones = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $opciones);
} catch (PDOException $e) {
    exit("Error al conectar con la base de datos: " . $e->getMessage());
}
```

2. Los tres ajustes obligatorios que configuran el éxito

ERRMODE — **ERRMODE_EXCEPTION**

Convierte cualquier error de SQL en una excepción capturable con `try/catch` en lugar de fallar silenciosamente.

FETCH — **FETCH_ASSOC**

Devuelve cada fila como un array asociativo limpio indexado por el nombre de columna (`$fila['email']`), ahorrando el doble de memoria.

¿Por qué desactivamos ATTR_EMULATE_PREPARES?

Si dejamos la emulación activa, PHP intenta simular los marcadores en lugar de enviar la consulta pura al motor MySQL. Al ponerlo en `false`, MariaDB/MySQL realiza la preparación nativa a nivel binario, garantizando máxima seguridad contra inyecciones y tipos de datos precisos.

CAPÍTULO 14 · EL BLINDAJE DEFINITIVO

Consultas Preparadas: Prepare y Execute

Separación radical entre la lógica de la sentencia SQL y los datos del usuario

3. Lectura con fetch() y fetchAll()

Para consultar registros, preparamos la plantilla con `$pdo->prepare()`, pasamos los parámetros seguros en un array a `execute()`, y recogemos los resultados:

consultas_pdo.php - Visual Studio Code

PHP consultas_pdo.php

```php
<?php
// Caso A: Obtener una única fila (ej: buscar alumno por ID)
$stmt = $pdo->prepare("SELECT id, nombre, email FROM alumnos WHERE id = :id");
$stmt->execute(['id' => $_GET['id'] ?? 0]);
$alumno = $stmt->fetch(); // Devuelve un array o false si no existe

if ($alumno) {
    echo "Hola, " . htmlspecialchars($alumno['nombre']);
}

// Caso B: Obtener todas las filas (ej: catálogo completo)
$stmtAll = $pdo->prepare("SELECT titulo, precio FROM cursos WHERE precio <= :max");
$stmtAll->execute(['max' => 300]);
$cursos = $stmtAll->fetchAll(); // Devuelve un array bidimensional
```

4. Escritura: INSERT, UPDATE y DELETE con marcadores seguros

Para modificar datos, el flujo es idéntico. Además, podemos conocer cuántas filas se vieron afectadas con `rowCount()` y el último ID generado con `lastInsertId()`:

escritura_pdo.php - Visual Studio Code

PHP escritura_pdo.php

```php
<?php
// Insertar un nuevo alumno de forma segura
$sql = "INSERT INTO alumnos (nombre, email, curso_id) VALUES (:nom, :mail, :curso)";
$stmt = $pdo->prepare($sql);
$stmt->execute([
    'nom'   => 'Lucía Fernández',
    'mail'  => 'lucia@alpe.es',
    'curso' => 1
]);

$nuevoId = $pdo->lastInsertId();
echo "Alumno registrado con éxito. Su nuevo ID es: " . $nuevoId;
```

La regla de oro de Ramón: Marcadores con nombre (:param)

Podéis usar interrogaciones `?` como marcadores posicionales, pero en mis clases siempre exijo **marcadores con nombre** (`:nom, :email`). Si tenéis un formulario con 12 campos y usáis signos de interrogación, equivocar el orden del séptimo parámetro causará un desastre silencioso. Con nombres, el código es autoexplicativo y a prueba de errores.

CAPÍTULO 14 · NIVEL PROFESIONAL

Transacciones en PDO: Todo o Nada

Garantizando la integridad atómica con beginTransaction, commit y rollBack

5. La analogía de la transferencia bancaria

Imaginad que hacéis una compra en una tienda online: se debe restar 100€ de vuestra cuenta y sumar 100€ a la del vendedor. ¿Qué ocurre si se ejecuta la primera consulta pero el servidor se apaga antes de la segunda? Habríais perdido 100€ en el limbo. Una **transacción** garantiza el principio ACID (Atomicidad): o se cumplen *todas* las operaciones, o el sistema revierte al estado original sin tocar nada.

transaccion_pdo.php - Visual Studio Code

PHP transaccion_pdo.php

```php
<?php
try {
    $pdo->beginTransaction(); // 1. Abrimos el escudo de la transacción

    // Operación A: Restar saldo al cliente
    $stmt1 = $pdo->prepare("UPDATE cuentas SET saldo = saldo - :monto WHERE id = :origen");
    $stmt1->execute(['monto' => 100, 'origen' => 1]);

    // Operación B: Sumar saldo al destinatario
    $stmt2 = $pdo->prepare("UPDATE cuentas SET saldo = saldo + :monto WHERE id = :destino");
    $stmt2->execute(['monto' => 100, 'destino' => 2]);

    $pdo->commit(); // 2. Ambas se ejecutaron bien: consolidamos los cambios
    echo "Transferencia completada con éxito.";

} catch (Exception $e) {
    $pdo->rollBack(); // 3. Algo falló: deshecho total automático
    echo "Error en la operación. Se ha cancelado todo: " . $e->getMessage();
}
```

Autoevaluación: ¿Por qué PDO::query() no debe usarse con variables del usuario?

> **Respuesta docente:** El método `$pdo->query()` ejecuta sentencias directas sin compilar. Solo es aceptable para consultas estáticas sin ningún dato externo (por ejemplo, `SELECT * FROM categorias ORDER BY orden`). En cuanto intervenga un valor de formulario o URL, el uso obligatorio es `prepare()` y `execute()`.

Fuentes y Documentación Oficial

Manual oficial de PHP: Extensión PDO y controladores: *php.net/manual/es/book.pdo.php*.
Sentencias preparadas en PHP PDO: *php.net/manual/es/pdo.prepared-statements.php*.

CAPÍTULO 15 · BLOQUE V: BASES DE DATOS

Proyecto CRUD Profesional: De la teoría a la realidad

Estructura limpia, listado paginado con buscador y creación con validación estricta

Ha llegado la hora de unir todas las piezas que hemos aprendido hasta ahora. El término **CRUD** corresponde al acrónimo en inglés de las cuatro operaciones fundamentales sobre cualquier entidad: **C**reate (crear), **R**ead (leer/listar), **U**pdate (editar) y **D**elete (borrar). En este capítulo construiremos el catálogo de cursos de nuestra academia con un código limpio y seguro.

1. Arquitectura de carpetas del proyecto

Para que vuestro código no parezca un plato de espaguetis desordenado, separaremos la configuración, la lógica y las plantillas visuales:

config/ **config/db.php**
Instancia singleton o archivo centralizado de conexión PDO que se incluye en todos los scripts del sistema.

public/ **index.php, crear.php, editar.php**
Controladores que reciben la petición HTTP, validan los datos y ejecutan las consultas preparadas.

2. Listado inteligente: Buscador y paginación con LIMIT y OFFSET

Si una tabla tiene 50.000 registros, jamás se debe hacer un `SELECT * FROM productos`. Tumbariáis el servidor por falta de memoria. La paginación profesional se realiza así:

index.php (Listado con Paginación) - Visual Studio Code

PHP index.php

```php
<?php
require_once __DIR__ . '/config/db.php';

$porPagina = 5;
$paginaActual = max(1, (int)($_GET['pagina'] ?? 1));
$offset = ($paginaActual - 1) * $porPagina;
$busqueda = trim($_GET['q'] ?? '');

// Consulta con filtro dinámico y paginación matemática
$sql = "SELECT * FROM cursos WHERE titulo LIKE :q ORDER BY id DESC LIMIT :limit OFFSET :offset";
$stmt = $pdo->prepare($sql);
$stmt->bindValue(':q', "%$busqueda%", PDO::PARAM_STR);
$stmt->bindValue(':limit', $porPagina, PDO::PARAM_INT);
$stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
$stmt->execute();
$cursos = $stmt->fetchAll();
```

¡Cuidado con bindValue() en LIMIT y OFFSET!
Si pasáis un array asociativo directamente a `execute(['limit' => $porPagina])`, PDO convertirá el entero en una cadena de texto entre comillas (`LIMIT '5'`), lo cual provoca un error de sintaxis en ciertas versiones de MySQL. Para el `LIMIT` y `OFFSET` debéis usar `bindValue()` con el tipo explícito `PDO::PARAM_INT`.

CAPÍTULO 15 · EL CICLO COMPLETO

Edición y Borrado: Las dos operaciones críticas

Formulario de edición precargado y por qué jamás se debe borrar por método GET

3. Edición con precarga de datos (UPDATE)

Para editar, primero consultamos el registro mediante su ID para pintar los campos en el formulario, y al recibir el `POST` realizamos la actualización:

editar.php - Visual Studio Code

PHP editar.php

```php
<?php
$id = (int)($_GET['id'] ?? 0);
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $titulo = trim($_POST['titulo'] ?? '');
    $precio = (float)($_POST['precio'] ?? 0);

    $update = $pdo->prepare("UPDATE cursos SET titulo = :t, precio = :p WHERE id = :id");
    $update->execute(['t' => $titulo, 'p' => $precio, 'id' => $id]);
    header('Location: index.php?mensaje=actualizado');
    exit;
}
$stmt = $pdo->prepare("SELECT * FROM cursos WHERE id = :id");
$stmt->execute(['id' => $id]);
$curso = $stmt->fetch() ?: exit('Curso no encontrado');
```

4. El peligro de borrar por GET y la solución profesional por POST

Muchos tutoriales de novatos ponen enlaces como `<a href="borrar.php?id=5">Eliminar</a>`. ¡Jamás hagáis esto en una aplicación real! Si un bot de Google o un pre-cargador de enlaces del navegador rastrea la página, **visitará todos los enlaces GET y borrará toda la base de datos automáticamente**. El borrado siempre debe exigir una petición `POST` con confirmación y token CSRF:

eliminar_seguro.php - Visual Studio Code

PHP eliminar_seguro.php

```php
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // 1. Verificar token CSRF
    if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
        exit('Token CSRF no válido');
    }
    $del = $pdo->prepare("DELETE FROM cursos WHERE id = :id");
    $del->execute(['id' => (int)$_POST['id']]);
    header('Location: index.php?mensaje=eliminado');
    exit;
}
```

Fuentes y Documentación Oficial

Patrones de diseño de persistencia de datos (DAO y Active Record): *martinfowler.com/eaaCatalog/*.
Guía de buenas prácticas con PHP y MySQL de PHP: The Right Way: *phptherightway.com/#databases*.

CAPÍTULO 16 · BLOQUE VI: POO Y MVC

Programación Orientada a Objetos: Clases y Objetos

Modelando el mundo real: encapsulación, $this y la revolución del Constructor Promotion

Hasta ahora hemos escrito código de arriba hacia abajo (procedimental). Pero cuando un proyecto supera las 1.000 líneas, mantener arrays asociativos sueltos se vuelve inmanejable. La **Programación Orientada a Objetos (POO)** nos permite empaquetar datos (propiedades) y comportamientos (métodos) en moldes llamados **Clases**.

1. El molde (Clase) frente a la galleta (Objeto)

Una **Clase** es el plano de diseño o el molde de galletas: define qué características y qué acciones tendrá una entidad. Un **Objeto** o *instancia* es la galleta horneada concreta con sus datos reales:

Molde **Clase Alumno**
Define que todo alumno tiene un nombre, un email y una función para calcular su nota media.

Galleta **Objeto $carlos**
Instancia concreta: nombre 'Carlos Gómez', email 'carlos@alpe.es', nota media de 8.5.

2. Constructor Promotion en PHP 8: El adiós al código repetitivo

En PHP 7 debíamos declarar las propiedades arriba, recibirlas en el constructor y asignarlas con `$this->nombre = $nombre`. En PHP 8, podemos declarar y asignar las propiedades directamente en los parámetros del constructor:

Alumno.php (PHP 8 Moderno) - Visual Studio Code

PHP Alumno.php

```php
<?php
declare(strict_types=1);

class Alumno {
    // Constructor Promotion de PHP 8: declara y asigna en 1 sola línea
    public function __construct(
        public readonly int $id,
        public string $nombre,
        public string $email,
        private float $nota = 0.0
    ) {}

    public function haAprobado(): bool {
        return $this->nota >= 5.0;
    }
}
```

Propiedades readonly: Inmutabilidad garantizada

Fijaos en `public readonly int $id`. Al marcar una propiedad como `readonly` en PHP 8.1+, solo puede inicializarse una única vez en el constructor. Si alguien intenta hacer `$alumno->id = 99;` más tarde, PHP arrojará un error fatal. ¡Seguridad matemática para claves primarias!

CAPÍTULO 16 · MODIFICADORES DE ACCESO

Encapsulación: Public, Private y Protected

Cómo proteger el estado interno de tus objetos y evitar manipulaciones accidentales

3. Los tres niveles de visibilidad

La **encapsulación** es el principio que dice: «No dejes que cualquiera toque las tripas de tu objeto». En su lugar, ofrecemos métodos públicos (getters y setters o métodos de dominio) para interactuar de forma controlada:

Modificador	Dónde se puede acceder	Analogía del restaurante de Ramón
`public`	Desde cualquier lugar (dentro y fuera de la clase).	La carta de platos del restaurante visible para cualquier comensal.
`private`	Únicamente desde el interior de la propia clase.	La receta secreta del chef; ningún camarero ni cliente puede verla.
`protected`	Desde la propia clase y desde sus clases hijas (herencia).	Los secretos de familia compartidos únicamente con los hijos del chef.

CuentaBancaria.php - Visual Studio Code

PHP CuentaBancaria.php

```php
<?php
class CuentaBancaria {
    public function __construct(
        public readonly string $iban,
        private float $saldo = 0.0
    ) {}

    public function ingresar(float $cantidad): void {
        if ($cantidad <= 0) throw new InvalidArgumentException('Cantidad no válida');
        $this->$saldo += $cantidad;
    }
    public function getSaldo(): float { return $this->saldo; }
}
```

Fuentes y Documentación Oficial

Manual oficial de PHP: Clases y Objetos en PHP 8: *php.net/manual/es/language.oop5.php*.
Promoción de propiedades en constructor de PHP 8.0: *php.net/releases/8.0/es.php#constructor-property-promotion*.

CAPÍTULO 17 · BLOQUE VI: POO Y MVC

Herencia, Polimorfismo e Interfaces

Reutilización de código con extends, contratos mediante implements y composición con Traits

¿Qué ocurre cuando tenemos diferentes tipos de usuarios (Alumno, Profesor, Administrador)? Todos tienen nombre y email, pero el profesor cobra nómina y el alumno tiene notas. En vez de copiar código, usamos **herencia** (`extends`) e **interfaces** (`implements`) para diseñar jerarquías limpias.

1. Herencia con extends y parent::__construct()

Herencia.php - Visual Studio Code

PHP Herencia.php

```php
<?php
abstract class Usuario {
    public function __construct(public string $nombre, public string $email) {}
    abstract public function obtenerRol(): string;
}

class Profesor extends Usuario {
    public function __construct(string $nom, string $mail, public string $especialidad) {
        parent::__construct($nom, $mail);
    }
    public function obtenerRol(): string { return "Docente en " . $this->especialidad; }
}
```

2. Interfaces: El contrato sagrado del polimorfismo

Una **interfaz** no tiene código; solo declara métodos que las clases firmantes *están obligadas a implementar*. Esto permite que una función reciba cualquier clase que cumpla el contrato:

NotificadorInterface.php - Visual Studio Code

PHP NotificadorInterface.php

```php
<?php
interface NotificadorInterface {
    public function enviar(string $destinatario, string $mensaje): bool;
}

class EmailNotificador implements NotificadorInterface {
    public function enviar(string $dest, string $msg): bool {
        return mail($dest, "Notificación Alpe", $msg);
    }
}
```

Fuentes y Documentación Oficial

Manual oficial de PHP: Herencia de objetos: *php.net/manual/es/language.oop5.inheritance.php.*
Interfaces de objetos y Traits en PHP: *php.net/manual/es/language.oop5.interfaces.php.*

CAPÍTULO 18 · BLOQUE VI: POO Y MVC

Namespaces, Composer y Autoloading PSR-4

Adiós a los require manuales infinitos y bienvenidos al ecosistema profesional moderno

¿Os imagináis tener que escribir 50 líneas de `require_once` al principio de cada fichero para cargar cada una de vuestras clases? Sería una pesadilla insostenible. En este capítulo aprenderemos a usar **namespaces** (espacios de nombres) para evitar colisiones y **Composer** con el estándar **PSR-4** para cargar automáticamente cualquier clase al instante.

1. ¿Qué es un Namespace? El sistema de carpetas lógicas

Si tenéis dos clases llamadas `Usuario` (una para la base de datos y otra de una librería externa), PHP dará un error fatal por duplicidad de nombre. El `namespace` resuelve esto actuando como un apellido o una carpeta virtual:

src/Models/Usuario.php - Visual Studio Code

PHP Usuario.php

```php
<?php
namespace App\Models;

class Usuario {
    public function __construct(public string $nombre) {}
}
```

2. Composer y el archivo composer.json con PSR-4

Composer es el gestor de dependencias oficial de PHP. En la raíz de nuestro proyecto creamos un archivo `composer.json` configurando el autoloading PSR-4:

composer.json - Visual Studio Code

JSON composer.json

```json
{
  "name": "alpe/academia-app",
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  }
}
```

Luego ejecutamos `composer dump-autoload` en la terminal. A partir de ese momento, un único `require_once 'vendor/autoload.php';` en nuestro punto de entrada cargará automáticamente cualquier clase según la necesitemos.

Fuentes y Documentación Oficial

Guía oficial de Composer: *getcomposer.org/doc/*.
Estándar de Autoloading PHP-FIG PSR-4: *php-fig.org/psr/psr-4/*.

CAPÍTULO 19 · BLOQUE VI: POO Y MVC

Arquitectura MVC desde Cero: La Separación Sagrada

Modelo, Vista y Controlador: la estructura sobre la que se asientan Laravel y Symfony

> ¿Qué significa MVC? Son las siglas de **Modelo**, **Vista** y **Controlador**. Es el patrón de arquitectura rey en el desarrollo web moderno. Su objetivo es simple pero tajante: *nunca mezclar consultas a la base de datos con etiquetas HTML ni con la lógica de control*. Vamos a construir uno funcional desde los cimientos.

1. Las tres patas del taburete MVC

M **El Modelo (Model)**
Gestiona los datos y las reglas del negocio. Habla con MariaDB mediante PDO y desconoce por completo la existencia de HTML o del navegador.

V **La Vista (View)**
La presentación visual. Solo contiene HTML y pequeñas etiquetas de salida como `<?= $titulo ?>`. No toma decisiones lógicas.

C **El Controlador (Controller)**
El cerebro orquestador. Recibe la petición del usuario, llama al Modelo para obtener datos y se los inyecta a la Vista correspondiente.

Router **Front Controller & Router**
Toda petición entra por `public/index.php`, que analiza la URL y delega en el Controlador adecuado.

2. Implementación de un Controlador en PHP 8

CursoController.php - Visual Studio Code

PHP CursoController.php

```php
<?php
namespace App\Controllers;
use App\Models\CursoModel;

class CursoController {
    public function index(): void {
        $model = new CursoModel();
        $cursos = $model->obtenerTodos();
        // Renderizamos la vista pasándole los datos
        require __DIR__ . '/../../views/cursos/index.view.php';
    }
}
```

Fuentes y Documentación Oficial
Patrón Modelo-Vista-Controlador en arquitecturas web: *en.wikipedia.org/wiki/Model-view-controller.*
Principios de arquitectura limpia (Clean Architecture de Robert C. Martin).

CAPÍTULO 20 · BLOQUE VII: VANGUARDIA

PHP 8 Moderno: Enums, Readonly Classes y Nullsafe

Las características más potentes y elegantes que diferencian a un programador actual de uno estancado

PHP ha evolucionado a una velocidad de vértigo. Si alguien os dice que PHP es un lenguaje tosco o desordenado, es porque no lo toca desde 2010. Con las versiones 8.1, 8.2, 8.3 y 8.4, disponemos de herramientas de tipado estricto y seguridad sintáctica que compiten de tú a tú con TypeScript, C# o Java. ¡Vamos a ver las joyas de la corona!

1. Enumeraciones (Enums): Tipado estricto de valores permitidos

¿Cuántas veces habéis visto estados guardados como cadenas mágicas tipo `'pendiente'` o `'cancelado'` donde una simple errata tipográfica rompía el programa? Los **Enums** solucionan esto de raíz:

EstadoPedido.php - Visual Studio Code

PHP EstadoPedido.php

```php
<?php
declare(strict_types=1);

enum EstadoPedido: string {
    case PENDIENTE = 'pen';
    case ENVIADO   = 'env';
    case ENTREGADO = 'ent';
}

function actualizarEstado(EstadoPedido $estado): void {
    echo "El pedido ahora está en: " . $estado->name . " (código " . $estado->value . ")";
}
```

2. Clases Readonly y Operador Nullsafe (?->)

8.2 **readonly class**

Convierte automáticamente todas las propiedades de una clase en inmutables. Ideal para Data Transfer Objects (DTOs) de base de datos o APIs.

?-> **Operador Nullsafe**

Evita anidar cinco `if ($obj !== null)`: `$pais = $usuario?->getDireccion()?->getPais();`. Si alguno es null, devuelve null sin lanzar error.

Fuentes y Documentación Oficial

Novedades oficiales de PHP 8.2 y 8.3: *php.net/releases/8.2/es.php* y *php.net/releases/8.3/es.php*.
Guía de enumeraciones en PHP: *php.net/manual/es/language.enumerations.php*.

CAPÍTULO 21 · BLOQUE VII: CONECTIVIDAD

Creación y Consumo de APIs REST con JSON

Servir datos en formato JSON, recibir payloads POST y consumir servicios externos con cURL

Hoy en día casi ninguna aplicación vive aislada: vuestra app PHP necesitará alimentar una app móvil en Flutter o React Native, o consultar la pasarela de pagos de Stripe o el tiempo de OpenWeather. En este capítulo aprenderéis a hablar el idioma de internet: **JSON y APIs REST**.

1. Construyendo un Endpoint JSON profesional en PHP

api/cursos.php - Visual Studio Code

PHP api/cursos.php

```php
<?php
declare(strict_types=1);
header('Content-Type: application/json; charset=utf-8');

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $cuerpoJson = file_get_contents('php://input');
    $datos = json_decode($cuerpoJson, true);

    if (!$datos || empty($datos['titulo'])) {
        http_response_code(400);
        echo json_encode(['error' => 'El campo título es obligatorio']);
        exit;
    }
    http_response_code(201);
    echo json_encode(['status' => 'creado', 'curso' => $datos['titulo']]);
}
```

2. Consumo de APIs externas con cURL

consumir_api.php - Visual Studio Code

PHP consumir_api.php

```php
<?php
$ch = curl_init('https://api.ejemplo.com/v1/alumnos');
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_TIMEOUT, 5);
$respuesta = curl_exec($ch);
$codigoHttp = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);

$datos = json_decode($respuesta, true);
```

Fuentes y Documentación Oficial

Documentación oficial de cURL en PHP: *php.net/manual/es/book.curl.php*.
Especificación HTTP y REST por Fielding: *ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm*.

CAPÍTULO 22 · CIERRE DEL MANUAL

Calidad Profesional: Depuración, PSR-12 y PHPStan

Manejo global de excepciones, análisis estático de código y las conclusiones de Ramón Abramo

¡Enhorabuena! Habéis llegado al capítulo final de este manual. Ya no sois unos novatos asustados ante una pantalla en negro: conocéis el ciclo completo HTTP, la sintaxis moderna de PHP 8, la seguridad con PDO y sentencias preparadas, la POO y las APIs REST. Para rematar vuestra formación, vamos a ver cómo garantizar que vuestro código tenga la calidad exigida por las empresas punteras.

1. Las tres herramientas de calidad profesional

PSR-12 **Estándar de Estilo PSR-12**

Define dónde van las llaves, cuántos espacios indentar y cómo nombrar métodos. Con PHP-CS-Fixer se formatea automáticamente en VS Code.

PHPStan **Análisis Estático con PHPStan**

Lee vuestro código sin ejecutarlo y detecta llamadas a métodos inexistentes, tipos incompatibles y variables posiblemente nulas.

2. Manejo Global de Excepciones: Jamás una pantalla blanca al cliente

bootstrap.php - Visual Studio Code

PHP bootstrap.php

```php
<?php
set_exception_handler(function (Throwable $e) {
    // 1. Registrar en fichero seguro en el servidor
    error_log("[" . date('Y-m-d H:i:s') . "] " . $e->getMessage() . " en " . $e->getFile());
    // 2. Mostrar mensaje amigable al usuario
    http_response_code(500);
    echo "<h1>Ha ocurrido un problema técnico. Estamos trabajando en ello.</h1>";
});


```

Las palabras de despedida de Ramón Abramo para sus alumnos

En mis más de 25 años formando a futuros programadores en Alpe Formación, he comprobado una y otra vez que el buen desarrollador no es el que memoriza todas las funciones de memoria, sino el que sabe **pensar con lógica, entender la arquitectura y ser constante**. Nunca tengáis miedo a que un script falle: cada error que veis en la consola es una oportunidad para aprender cómo funcionan las cosas por dentro. Seguid practicando, construid proyectos personales y disfrutad de este maravilloso oficio que es la programación. ¡Un abrazo muy fuerte y a por todas!

Fuentes y Documentación Oficial

Guía de estilo PHP-FIG PSR-12: *php-fig.org/psr/psr-12/*.
Herramienta de análisis estático PHPStan: *phpstan.org/user-guide/getting-started*.

ANEXO A · LA HOJA DE RESCATE

Chuleta Rápida de Sintaxis, Superglobales y HTTP

Todo lo que necesitas tener a mano mientras programas en tu día a día

1. Las Superglobales de PHP de un Vistazo

Variable	Origen / Utilidad	Ejemplo de acceso seguro
`$_GET`	Datos enviados por la URL (query string). Para lecturas y búsquedas.	`$id = (int)($_GET['id'] ?? 0);`
`$_POST`	Datos enviados por formulario con método POST (ocultos en el cuerpo).	`$email = trim($_POST['email'] ?? '');`
`$_SESSION`	Datos de sesión persistentes guardados en el servidor para este usuario.	`$_SESSION['user_id'] = $user['id'];`
`$_COOKIE`	Pequeños datos almacenados en el navegador del cliente.	`$tema = $_COOKIE['tema'] ?? 'claro';`
`$_FILES`	Archivos binarios subidos al servidor mediante multipart/form-data.	`$_FILES['foto']['tmp_name']`
`$_SERVER`	Metadatos del servidor y de la petición HTTP actual.	`$_SERVER['REQUEST_METHOD'] === 'POST'`

2. Códigos de Estado HTTP Esenciales

200 OK
Petición procesada con éxito y respuesta devuelta normalmente.

201 Created
Nuevo recurso creado con éxito en el servidor (típico en POST de APIs).

400 Bad Request
Petición defectuosa: datos faltantes o que no superan la validación.

401 / 403 Unauthorized / Forbidden
401: falta login; 403: logueado pero sin permisos para esa acción.

404 Not Found
El recurso, registro de base de datos o ruta no existe en el sistema.

500 Internal Server Error
Fallo no capturado en PHP (excepción, error de base de datos o sintaxis).

3. Funciones Nativas Imprescindibles

Categoría	Función	Para qué sirve exactamente
Seguridad	`htmlspecialchars($str, ENT_QUOTES, 'UTF-8')`	Neutraliza inyecciones XSS transformando caracteres peligrosos en entidades HTML.
Seguridad	`password_hash($pass, PASSWORD_BCRYPT)`	Genera hash criptográfico seguro e irreversible con sal aleatoria.
Arrays	`count(), in_array(), array_key_exists()`	Contar elementos, buscar valor o comprobar si existe una clave asociativa.
Cadenas	`str_contains(), str_starts_with(), mb_strlen()`	Comprobaciones modernas de subcadenas y medición correcta de longitud con tildes.

ANEXO B · ENTORNO DE TRABAJO

Guía de Atajos y Extensiones para VS Code

Convierte tu editor en un entorno de desarrollo integrado ultrapotente

1. Las 5 Extensiones Obligatorias para el Alumno

Extensión	Autor	Por qué es imprescindible en clase
PHP Intelephense	Ben Mewburn	Autocompletado inteligente de clases, métodos, firmas de tipos, importación automática de namespaces y detección de errores sintácticos en vivo.
PHP CS Fixer	junstyle	Aplica automáticamente el estándar de estilo PSR-12 al guardar el archivo (`Ctrl + S`).
Error Lens	Alexander	Muestra los errores y advertencias de PHP directamente sobre la línea de código en rojo o amarillo, sin esperar a abrir la pestaña de problemas.
Database Client / HeidiSQL	cweijan	Permite conectarse a MariaDB/MySQL directamente desde la barra lateral de VS Code para consultar tablas sin salir del editor.
GitLens	GitKraken	Muestra quién escribió cada línea de código y cuándo, facilitando el control de versiones en proyectos de equipo.

2. Los 8 Atajos de Teclado que Multiplican tu Velocidad

Alt + Shift + ↓ — **Duplicar Línea**
Duplica la línea o selección actual hacia abajo al instante sin cortar ni pegar.

Alt + ↑ / ↓ — **Mover Línea**
Sube o baja la línea de código completa de posición limpiamente.

F2 — **Renombrado Simbólico**
Cambia el nombre de una variable o función en TODOS los archivos del proyecto a la vez.

Ctrl + D — **Selección Múltiple**
Selecciona la siguiente aparición de la misma palabra para editar varias a la vez.

Ctrl + P — **Navegación Rápida**
Abre cualquier archivo del proyecto escribiendo solo un par de letras de su nombre.

Ctrl + ñ / ` — **Terminal Integrada**
Abre y oculta la terminal de PowerShell sin cambiar de ventana.

Shift + Alt + F — **Formatear Documento**
Indenta y embellece todo el archivo según las directrices PSR-12.

Ctrl + / — **Comentar / Descomentar**
Convierte la línea seleccionada en comentario de PHP (`//`) o viceversa.

ANEXO C · BOTIQUÍN DE EMERGENCIAS

Los Errores Típicos del Principiante y su Solución

Caza los fallos más habituales en segundos y ahórrate horas de desesperación

1. Parse error: syntax error, unexpected token ";" o "{"

Causa: Casi siempre se debe a que te has olvidado de cerrar un paréntesis en la línea *anterior*, has puesto una coma en vez de un punto y coma, o falta una llave de cierre `}`.
Solución de Ramón: Mira siempre la línea señalada *y la línea inmediatamente superior*.

2. Warning: Undefined array key "nombre"

Causa: Intentas leer `$_POST['nombre']` o `$fila['campo']` pero la clave no existe porque el usuario aún no envió el formulario o el nombre en el HTML era `name="name"`.
Solución de Ramón: Usa siempre el operador null coalescing: `$nombre = $_POST['nombre'] ?? '';`.

3. Warning: Cannot modify header information - headers already sent

Causa: Has ejecutado un `header('Location: ...')` o un `session_start()` después de haber enviado HTML, un `echo` o incluso un espacio en blanco antes de la etiqueta `<?php`.
Solución de Ramón: Las redirecciones y sesiones van **al principio absoluto** del fichero, antes de cualquier HTML.

4. Fatal error: Uncaught TypeError: Unsupported operand types: string + int

Causa: Vienes de JavaScript y has usado el operador suma `+` para concatenar texto (`$msg = "Hola " + $nombre;`). En PHP el `+` es exclusivamente matemático.
Solución de Ramón: En PHP se concatena con el punto: `$msg = "Hola " . $nombre;`.

ANEXO C · BOTIQUÍN DE EMERGENCIAS

Los Errores Típicos en Bases de Datos y POO

Soluciones rápidas a fallos de PDO, claves foráneas y llamadas a métodos

5. Fatal error: Call to a member function prepare() on null

Causa: Intentas hacer `$pdo->prepare()` dentro de una función o método de clase, pero la variable `$pdo` no está en el ámbito (scope) local de la función.
Solución de Ramón: Pasa `$pdo` como parámetro en el constructor de tu clase (inyección de dependencias) o utiliza una clase de conexión estática.

6. SQLSTATE[23000]: Integrity constraint violation: Cannot add or update child row

Causa: Intentas insertar un registro hijo asignándole un `curso_id` que no existe en la tabla padre `cursos`.
Solución de Ramón: Comprueba primero que el ID padre existe o valida en el formulario antes de ejecutar el INSERT.

7. Class "App\Models\Usuario" not found

Causa: El nombre del archivo en el disco no coincide exactamente en mayúsculas/minúsculas con el nombre de la clase (en Linux los nombres son sensibles a mayúsculas), o no has ejecutado `composer dump-autoload` tras añadir la clase.
Solución de Ramón: Revisa la regla PSR-4: `Usuario.php` debe llamarse con U mayúscula y estar en `src/Models/`.

8. El formulario se envía pero los campos llegan vacíos

Causa: Has puesto `<input id="email">` en lugar de `<input name="email">`. El atributo `id` solo sirve para CSS y JavaScript; PHP **únicamente lee el atributo name**.

ANEXO D · VOCABULARIO DE LA INDUSTRIA

Glosario de Términos para el Futuro Desarrollador

El diccionario técnico indispensable para hablar con propiedad en el entorno laboral

ACID — **Atomicidad, Consistencia, Aislamiento y Durabilidad**
Las 4 propiedades que garantizan que las transacciones en una base de datos relacional son seguras y no se corrompen.

CSRF — **Cross-Site Request Forgery**
Ataque que engaña al navegador de un usuario autenticado para ejecutar acciones no deseadas. Se frena con tokens secretos.

DSN — **Data Source Name**
Cadena de texto que describe la ubicación, nombre de BD, driver y juego de caracteres para que PDO se conecte.

DTO — **Data Transfer Object**
Objeto simple sin lógica de negocio cuya única misión es transportar datos tipados de una capa a otra (ej: una clase readonly).

MVC — **Modelo-Vista-Controlador**
Patrón arquitectónico que desacopla la persistencia de datos (Modelo), la interfaz visual (Vista) y la orquestación (Controlador).

PDO — **PHP Data Objects**
Capa de abstracción orientada a objetos nativa de PHP para conectar y ejecutar consultas preparadas seguras en bases de datos.

PSR — **PHP Standards Recommendation**
Estándares técnicos oficiales propuestos por PHP-FIG (PSR-4 para autoloading, PSR-12 para estilo de código).

XSS — **Cross-Site Scripting**
Inyección de código JavaScript malicioso en la pantalla de otros usuarios. Se neutraliza con `htmlspecialchars()`.

Fuentes y Documentación Oficial

Glosario oficial del Manual de PHP: *php.net/manual/es/indexes.php*.
Glosario de seguridad web de OWASP: *owasp.org/www-community/*.

ALPE FORMACIÓN · SANTANDER

Aprende el PHP que demandan las empresas punteras

¿Cansado de tutoriales desactualizados que enseñan prácticas obsoletas de hace quince años? Este manual ha sido diseñado desde cero en las aulas de **Alpe Formación** por **Ramón Abramo**, docente con más de un cuarto de siglo de experiencia formando a promociones de programadores que hoy trabajan en empresas punteras de toda España y Europa.

¿Qué encontrarás en este manual completo?

- **Arquitectura Web sin secretos:** La analogía del restaurante para entender servidores, clientes y HTTP.
- **Sintaxis PHP 8 Moderna:** Tipado estricto, constructor promotion, enums y nullsafe.
- **Seguridad Real contra Hackers:** Blindaje contra inyección SQL, CSRF tokens y contraseñas seguras con bcrypt.
- **Bases de Datos con PDO:** Sentencias preparadas nativas, transacciones atómicas y CRUD paginado.
- **Arquitectura MVC y Composer:** Patrones profesionales, PSR-4 y consumo de APIs REST con JSON y cURL.
- **Anexos y Chuletas:** Botiquín de emergencias con los 20 errores típicos y atajos de VS Code.

Ramón Abramo

Docente y formador tecnológico con más de 25 años de trayectoria profesional en Alpe Formación. Especialista en arquitecturas web, PHP, JavaScript y bases de datos relacionales.

Alpe Formación · Pasaje de Peña 1, Santander (Cantabria)

Edición Oficial 2026